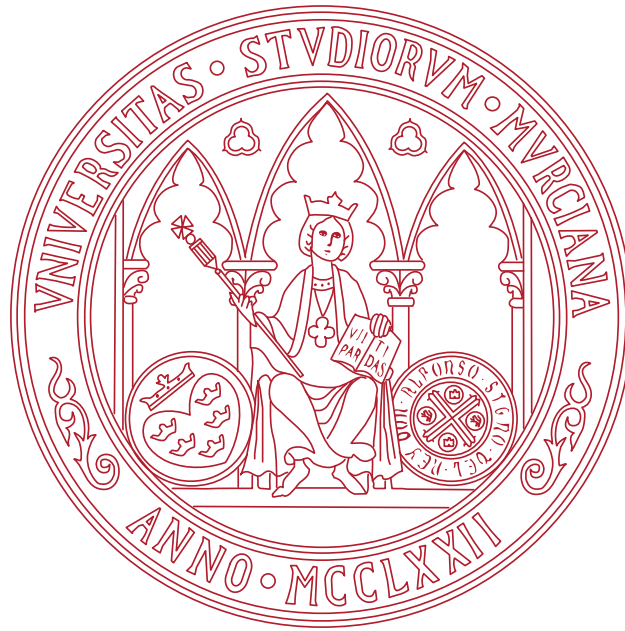




UNIVERSIDAD DE MURCIA

TRABAJO FIN DE GRADO

Impacto de los sistemas de memoria en grandes modelos de lenguaje

Estudiante:

Javier Patricio LUJÁN ROMERO
jp.lujanromero@um.es

Tutor:

Eduardo MARTÍNEZ GRACIÁ
edumart@um.es

Mayo 2026

A mi familia por su apoyo incondicional desde el inicio.

Mi madre y su ternura.

Mi padre y su sacrificio.

Índice general

Resumen	IV
Extended abstract	v
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Estructura del documento	2
2. Estado del arte	3
2.1. Grandes modelos de lenguaje	3
2.2. La ventana de contexto y sus problemas	5
2.3. Sistemas de memoria	7
2.3.1. Escenarios de uso	8
2.3.2. Mem0	9
2.4. LongMemEval	11
2.5. Métricas de evaluación	13
2.5.1. Métricas compartidas	13
2.5.2. Métricas específicas del uso de un sistema de memoria	14
2.6. Cálculo del coste de CO ₂	15
3. Análisis de objetivos y metodología	17
3.1. Objetivos	17
3.2. Herramientas y tecnologías	18
3.2.1. Entorno de Programación	18
3.2.2. Entorno de desarrollo	19
3.2.3. Modelos de lenguaje	19
3.3. Metodología de trabajo	20
4. Diseño y resolución	22
4.1. Pipeline de evaluación	22
4.1.1. Implementación concreta	23
4.2. Evaluación con LongMemEval-Oracle	25
4.3. Evaluación con LongMemEval-S	30
4.4. Líneas de mejora del sistema de memoria	36

4.4.1. Refinamiento del <i>prompt</i> de generación	37
4.4.2. Uso de modelos de <i>embeddings</i> de mayor capacidad	38
4.4.3. Uso de modelos de lenguaje más potentes para la generación	39
5. Conclusiones y vías futuras	42
Bibliografía	45
Apéndices	46
A. Prompts utilizados en el proyecto	46
A.1. Sistemas de memoria	46
A.2. Prompts de los evaluadores	47
A.2.1. Evaluación binaria de correctitud (<i>LLMJudgeEvaluator</i>)	47
A.2.2. Exactitud de referencia (<i>ReferenceAccuracyEvaluator</i>)	50
A.3. Prompts de extracción de memorias	51

Índice de figuras

2.1.	Arquitectura de un Transformer. Extraído de [17].	4
2.2.	Visualización del proceso de tokenización. Fuente: Elaboración propia utilizando la herramienta Tokenizer [10] de OpenAI.	5
2.3.	Evolución de la ventana de contexto en los últimos años. Datos extraídos de OpenAI y MetaAI.	6
2.4.	Visualización de los dos escenarios de uso. Fuente: Elaboración propia. . .	9
2.5.	Pipeline de adición de memorias en Mem0. Extraído de [2].	10
2.6.	Pipeline de búsqueda de Mem0. Extraído de [2].	11
2.7.	Ejemplos de los tipos de preguntas de LongMemEval. Extraído de [18]. . .	12
4.1.	Esquema del pipeline de evaluación diseñado para este proyecto.	23

Resumen

En este proyecto se plantea el problema de...

Extended abstract

This project faces the problem of...

Introducción

1.1. Motivación

En los últimos años la IA, en particular los grandes modelos de lenguaje —LLMs, por sus siglas en inglés— han supuesto una revolución y un cambio de paradigma ya no solo dentro de la industria del Machine Learning, sino en todos los ámbitos de la sociedad. Las aplicaciones de chat basadas en estos grandes modelos de texto, como ChatGPT o Google Gemini, se han convertido en herramientas principales para millones de usuarios en todo el mundo, realizando una inmensidad de tareas distintas.

Estas tareas tienen diferentes niveles de complejidad, desde tareas simples que se resuelven con una respuesta directa, a tareas más complejas en las que el propio modelo debe replantear los conceptos con los que está trabajando y cómo los ha de usar para llegar a la solución.

Dentro de estas tareas complejas, se pueden plantear dos situaciones: una en la que el modelo dispone de toda la información necesaria para resolver la tarea en el prompt, y la complejidad radica en el proceso deductivo que el modelo debe realizar para llevarla a cabo exitosamente, como por ejemplo la resolución de problemas matemáticos avanzados. Por otro lado, tenemos el caso en el que la resolución exitosa de la tarea requiere de otra información adicional que no se encuentra en el prompt; como ejemplo de este tipo de tarea tenemos la modificación de un código fuente ajustándose a los estándares del proyecto en el que se encuentra.

Es dentro de este segundo tipo de tareas donde ha proliferado el uso de los conocidos agentes de IA: sistemas autónomos capaces de percibir su entorno, razonar sobre posibles cursos de acción y tomar y ejecutar decisiones [12]. Estos agentes tienen como base un LLM principal, el cual realiza el razonamiento, y se le dota de un conjunto diverso de herramientas que le permiten interactuar con su entorno. Ejemplos de estas herramientas son: navegadores web, RAG en bases de datos, editores de código e incluso otros modelos de IA especializados en diferentes tareas.

Pero para el desarrollo de este trabajo, nos vamos a centrar en un tipo de herramienta en concreto: los sistemas de memoria [2]. Estos sistemas permiten a los agentes almacenar y recuperar información relevante a lo largo del tiempo, mejorando su capacidad para manejar tareas que requieren conocimiento a largo plazo y reduciendo la necesidad de incluir grandes cantidades de texto en el prompt. De esta manera se construyen agentes más eficientes tanto a nivel computacional como en el uso de tokens, que por ende son más económicos y generan un menor impacto ambiental.

1.2. Objetivos

El objetivo principal del trabajo es realizar un estudio sobre el impacto que tienen los sistemas de memoria en el rendimiento de los LLMs en distintos términos: latencia, precisión y consumo de tokens.

Con el fin de realizar este primer objetivo se ha creado un pipeline modular, que permite integrar diferentes sistemas de memoria para LLMs, facilitando la experimentación y evaluación de estos sistemas en diversas tareas y datasets. Se cumple de esta manera el segundo objetivo, que consiste en diseñar un sistema accesible que permita evaluar diferentes sistemas de memoria en LLMs de manera sencilla y adaptable.

1.3. Estructura del documento

El trabajo se ha estructurado de acuerdo con las indicaciones dadas en la guía docente de la asignatura correspondiente al Trabajo de Fin de Grado. Por tanto, el resto del documento se organiza de la siguiente manera:

- En el capítulo 2 se realiza un estudio del estado del arte relativo a los conceptos tratados en este trabajo, abarcando desde la arquitectura detrás de los LLMs hasta los sistemas de memoria y las diferentes formas de evaluar su rendimiento.
- En el capítulo 3 se describen de manera detallada los objetivos del trabajo, se introducen las herramientas y tecnologías empleadas, y se explica la metodología seguida.
- En el capítulo 4, diseño y resolución, se expone la implementación realizada del pipeline que permite evaluar los sistemas de memoria. Además, se describen los experimentos realizados y los resultados obtenidos.
- Finalmente, en el capítulo 5 se exponen las conclusiones finales del trabajo, se discuten las limitaciones encontradas y se proponen posibles líneas de trabajo futuro.

Estado del arte

En este capítulo se realiza un estudio del estado del arte relativo a los conceptos sobre los que trata el trabajo. Se inicia con una introducción a los grandes modelos de lenguaje en la sección 2.1, permitiendo entender la arquitectura que tienen detrás y sus principales características. En la siguiente sección, la 2.2, se expone el concepto de la ventana de contexto y los principales problemas asociados a su uso. A continuación se presentan en la sección 2.3 los sistemas de memoria como posible solución a los problemas mencionados anteriormente. Posteriormente, en la sección 2.4 se describe LongMemEval, el dataset elegido como benchmark para evaluar el rendimiento de los sistemas de memoria. Finalmente, en la sección 2.5 se describen las métricas que se han usado para evaluar la diferencia de rendimiento entre el uso exclusivo de la ventana de contexto y el uso de un sistema de memoria.

2.1. Grandes modelos de lenguaje

Como se ha expuesto en la introducción, la IA y los LLMs son, cada vez más, un pilar fundamental en nuestra sociedad. Por tanto, es necesario entender, con cierto grado de profundidad, qué significa cada uno de estos conceptos y en qué se diferencian.

Por un lado, el concepto más general de todos es el de inteligencia artificial (IA). La IA es un campo de estudio de la informática cuyo objetivo es crear máquinas inteligentes capaces de mejorar de manera autónoma explorando y aprendiendo del mundo real [20]. Dentro de este campo encontramos el Aprendizaje Computacional, o Machine Learning, que es la rama que se encarga de la construcción de algoritmos que, a partir de un conjunto de datos de entrenamiento, son capaces de aprender patrones, lo que les permite posteriormente realizar predicciones o tomar decisiones sobre datos nuevos que no han visto antes; es decir, les permite generalizar [16]. Finalmente, es dentro del Machine Learning donde nos encontramos con el Aprendizaje Profundo, o Deep Learning, que se acerca por fin a la concepción generalizada de la IA.

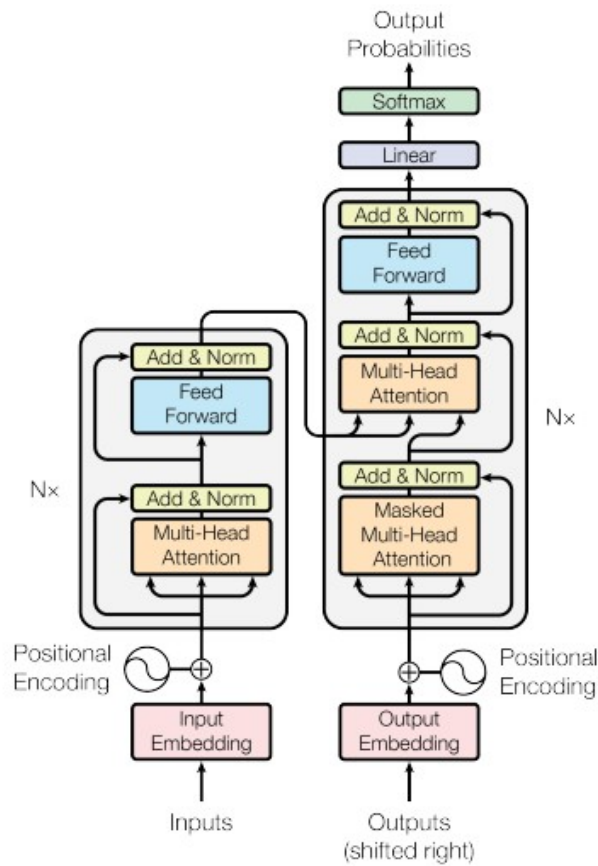


Figura 2.1: Arquitectura de un Transformer. Extraído de [17].

Este aprendizaje profundo se basa en redes neuronales: un modelo computacional inspirado en las neuronas biológicas del cerebro humano, que se comunican a través de señales eléctricas [16]. Esto se traduce en una serie de capas conectadas entre sí de perceptrones – un modelo matemático de neurona. La profundidad radica en el número de capas ocultas, que se encuentran entre la capa de entrada y la de salida; esto les permite aprender representaciones jerárquicas y complejas de los datos, mediante el uso de algún algoritmo que ajusta la fuerza de las conexiones entre las neuronas individuales de capas adyacentes, lo que se conoce como pesos [16].

En un principio, entrenar estos modelos de Deep Learning requería recursos computacionales significativos y bastante tiempo. Sin embargo, desde hace unos años se ha desarrollado una nueva arquitectura de red neuronal, los denominados **Transformers**, que ha revolucionado el campo del Deep Learning, especialmente en el procesamiento del lenguaje natural (NLP) [17]. Es gracias a esta arquitectura que se han podido crear los LLMs (Large Language Models), llamados así por la inmensa cantidad de datos con los que son entrenados y por su elevado número de parámetros, los valores numéricos ajustables que incluyen, entre otros, los pesos de las conexiones entre las neuronas.

Cabe, por tanto, finalizar esta sección con una breve explicación sobre esta arquitectura. Un Transformer es una arquitectura de red neuronal basada en un mecanismo de atención, que permite al modelo enfocarse en diferentes partes de la entrada de manera

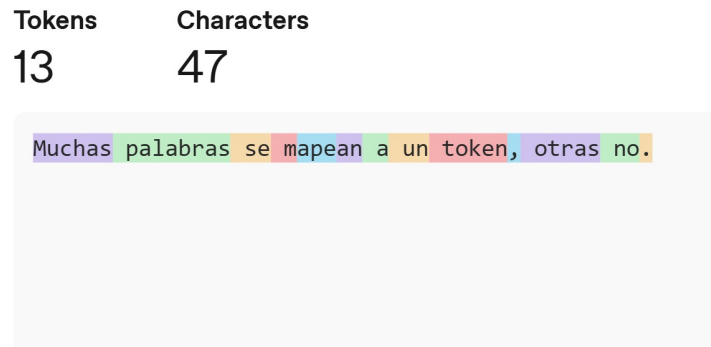


Figura 2.2: Visualización del proceso de tokenización. Fuente: Elaboración propia utilizando la herramienta Tokenizer [10] de OpenAI.

adaptativa, siendo por tanto altamente efectiva a la hora de procesar y generar secuencias de datos, como texto, audio o video; y que es además altamente paralelizable.

Los Transformers se componen principalmente de dos partes: el codificador (encoder) y el decodificador (decoder), la parte izquierda y derecha de la figura 2.1 respectivamente [17]. El codificador toma la secuencia de entrada previamente convertida en *embeddings*, representaciones numéricas, y la transforma en una representación interna que captura tanto la palabra como su contexto, mientras que el decodificador utiliza esta representación para generar la secuencia de salida, palabra por palabra de manera auto-regresiva, es decir, utilizando la salida generada previamente para generar la siguiente. Ambos componentes están formados por múltiples capas de atención y redes neuronales feedforward.

2.2. La ventana de contexto y sus problemas

En la sección anterior se ha explicado brevemente la arquitectura detrás de los Transformers; sin embargo, no se ha detallado cómo se introduce la secuencia de entrada en estos modelos. Es importante destacar que los modelos no trabajan con texto plano, sino que, como se ha indicado, toman como entrada *embeddings*. Estos *embeddings* son vectores de valores continuos que representan tanto a las palabras como su relación semántica con otras, y son las unidades matemáticas con las que opera el modelo [9].

Para llegar a generar estos *embeddings*, existe un paso previo fundamental: la **tokenización**. En este proceso, las palabras, caracteres individuales o subconjuntos de caracteres, son divididos en unidades mínimas de información llamadas *tokens* [21], tal como se muestra en la figura 2.2. Finalmente, cada uno de estos *tokens* es convertido en un primer *embedding* base para ser procesado por el modelo.

Los tokens constituyen, por tanto, la unidad básica de información con la que se introduce el texto en los modelos de lenguaje. Y al ser la base sobre la que se calcula la representación interna del texto, se utilizan como métrica estándar para establecer los límites de capacidad en la entrada de datos. Este límite se denomina de manera formal

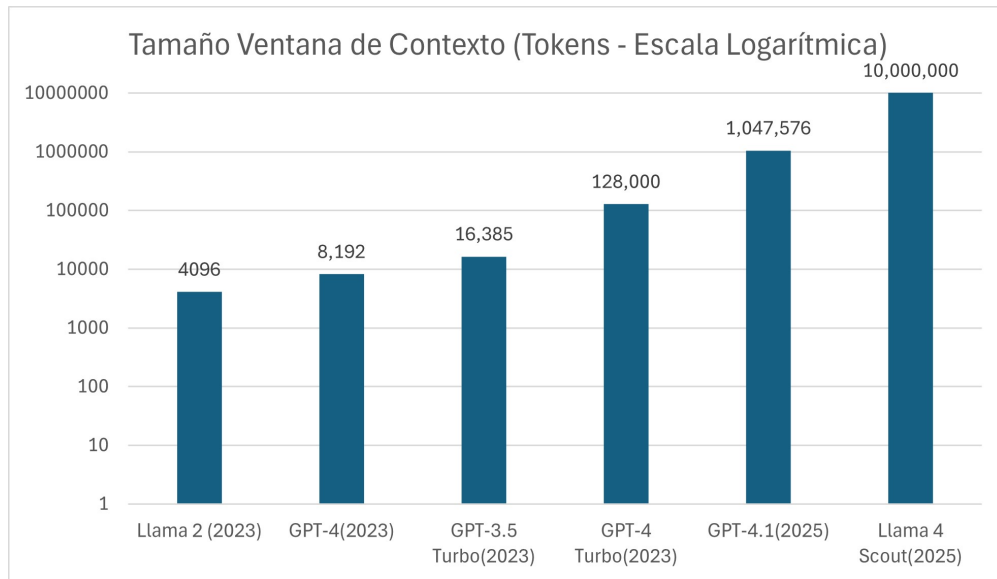


Figura 2.3: Evolución de la ventana de contexto en los últimos años. Datos extraídos de OpenAI y MetaAI.

ventana de contexto, y se refiere a la cantidad máxima de tokens que un modelo puede procesar en una sola instancia.

La longitud de la ventana de contexto ha sido un componente en el que se ha puesto especial énfasis y ha sufrido un aumento drástico en los últimos años (véase la figura 2.3) pasando de unos pocos miles de tokens a más de un millón en algunos modelos recientes [1]. Este aumento ha sido posible gracias a avances en la arquitectura de los modelos, como la introducción de mecanismos de atención más eficientes, lo que ha permitido a los modelos manejar contextos más largos sin una degradación significativa del rendimiento [1].

Pero, aunque como se acaba de ver, la tendencia actual de la industria es un progresivo aumento de la ventana de contexto, no se debe confiar exclusivamente en esta como único modelo para la gestión de la información, pues esta solución no está exenta de problemas, entre los que destacan los siguientes:

1. Un primer problema que tiene la ventana de contexto es que, como indica su propia definición, es, de antemano, un límite fijado [2]. Esto implica que, aunque se pueda aumentar la ventana de contexto, siempre existirá un límite máximo de tokens que el modelo pueda procesar, lo que puede resultar insuficiente para tareas que requieren trabajar con grandes cantidades de información o con conocimiento a largo plazo.
2. Por otro lado, aunque el contexto con el que se está trabajando quepa dentro de la ventana, procesar grandes cantidades de texto tiene un coste computacional significativo, lo que se traduce en un coste económico elevado [8]. Esto hace que, aunque se pueda aumentar la ventana de contexto, no sea una solución viable a largo plazo ni asumible en todas las aplicaciones, pues el coste de procesar grandes cantidades de texto deja de ser económicamente viable.

3. Además, aun ignorando tanto el coste computacional como el límite de tokens, la ventana de contexto tiene otra desventaja crucial: el rendimiento de los modelos de lenguaje, es decir, la capacidad de procesar y responder correctamente a preguntas sobre una entrada, tiende a degradarse a medida que aumenta su tamaño [3], siendo aún más pronunciada esta degradación cuando en el contexto se incluye información irrelevante para la tarea específica que se está realizando, lo que puede resultar en una disminución significativa del rendimiento del modelo [15].
4. Finalmente, el último desafío que se presenta es en la propia naturaleza del aprendizaje. La ventana de contexto permite retener información con la que trabajar, similar a la memoria de trabajo humana, pero es incapaz de procesar la información a lo largo del tiempo, e incluso de diferentes conversaciones y chats [11]. Esto limita las capacidades de un LLM para generalizar y aprender de la experiencia, pues no puede retener información a largo plazo, lo que se traduce en una falta de continuidad y coherencia en las conversaciones, y en una incapacidad para aprender de la experiencia pasada.

2.3. Sistemas de memoria

Como solución a estos problemas, se han propuesto diferentes enfoques, entre los que destacan los basados en memoria –*la habilidad para almacenar, actualizar y recuperar conocimiento* [19]. El objetivo de los mismos es reducir la necesidad de procesar grandes cantidades de texto, economizando así recursos computacionales, y a su vez, permitiendo mejorar el rendimiento en tareas para las que la ventana de contexto es insuficiente.

Para entender estos sistemas de memoria es necesario responder a una serie de preguntas clave: ¿de dónde viene la información que se va a almacenar en la memoria?, ¿qué se debe hacer con esta información? y ¿cómo se pueden almacenar dichas memorias?

En cuanto a las fuentes de la información, Zhang et al. [20] proponen tres categorías principales: la información de la tarea que se está realizando, la información de tareas que ya se han realizado, e información externa obtenida de herramientas. Mientras que la primera es la más relevante para la tarea actual, y la tercera permite expandir el conocimiento del modelo más allá de sus barreras iniciales, la segunda es crucial para permitir que el modelo aprenda de la experiencia, lo que es fundamental para lograr una memoria a largo plazo.

Una vez obtenida esta información, es necesario decidir qué hacer con ella y cómo llevarla de información a memoria. Para esto, se pueden identificar tres procesos clave: la proyección de observaciones directas a memorias concisas e informativas; la consolidación de memorias, procesando, organizando y asentando información para mejorar su calidad y relevancia; y la recuperación de memorias, que consiste en recuperar información importante de la memoria para apoyar la toma de decisiones y la generación de respuestas [20].

Finalmente, queda la cuestión de cómo almacenar las memorias para poder realizar las tres operaciones anteriores de manera eficiente. Para esto, se pueden clasificar los

sistemas de memoria en dos grandes categorías: memoria en forma textual y memoria en forma paramétrica [20].

La primera consiste en guardar las memorias en forma de texto, ya sea estructurado o no, e introducirlo en el contexto del agente, lo que permite una mayor flexibilidad y adaptabilidad a diferentes tareas, aunque a expensas de la eficiencia computacional y el sobrecoste de procesamiento. Por otro lado, la forma paramétrica consiste en guardar las memorias, mediante la actualización de los parámetros del modelo, lo que permite una *recuperación* más rápida y eficiente, pero a expensas de la interpretabilidad y la flexibilidad [20].

Debido a las ventajas que ofrece la memoria en forma textual, es un enfoque que ha sido ampliamente probado en diferentes sistemas de memoria como: MemoryBank [22], un sistema de memoria que imita a los humanos centrándose en almacenar la información necesaria para entender la personalidad del usuario; Zep [13], que introduce una capa de memoria basada en un grafo de tres niveles que abarca desde los datos en crudo hasta abstracciones semánticas; o CLIN [7], un sistema persistente y dinámico de memoria textual basado en frases que representan abstracciones causales obtenidas de la interacción del agente con el entorno a la hora de aprender a realizar diferentes tareas.

2.3.1. Escenarios de uso

Una vez introducidos los sistemas de memoria, vamos a proceder a describir los dos escenarios de uso que se van a comparar en este trabajo: el uso exclusivo de la ventana de contexto, y el uso de un sistema de memoria.

En ambos escenarios se asume que se está trabajando con un chat entre un usuario y un asistente de IA, y se evalúa la capacidad que tiene el asistente para responder a una pregunta concreta, cuya respuesta se encuentra en una conversación previa.

En el caso del **uso exclusivo de la ventana de contexto**, al asistente se le introduce como contexto toda la conversación previa, incluyendo tanto la información relevante para responder a la pregunta como la irrelevante, y se le pide que responda a la pregunta utilizando toda esta información.

En este escenario, se apreciarán claramente los problemas asociados a la ventana de contexto (2.2), permitiendo evaluar el rendimiento del modelo en escenarios donde la cantidad de información supera la capacidad de la ventana de contexto, analizar la degradación bajo la presencia de información irrelevante y evaluar el coste computacional asociado a procesar grandes cantidades de texto.

Por otro lado, en el caso del **uso de un sistema de memoria** se debe separar el proceso de respuesta en dos fases. Una primera fase que realiza la creación de memorias de la conversación, en la que se debe simular el proceso que hubiera seguido el sistema de memoria para crear las memorias durante la conversación, introduciendo al sistema de memoria la conversación en pares usuario-asistente, y obteniendo así las memorias que el sistema hubiera creado. Y una segunda fase de recuperación de memorias, donde se introduce la pregunta al sistema de memoria, y este devuelve las memorias relevantes

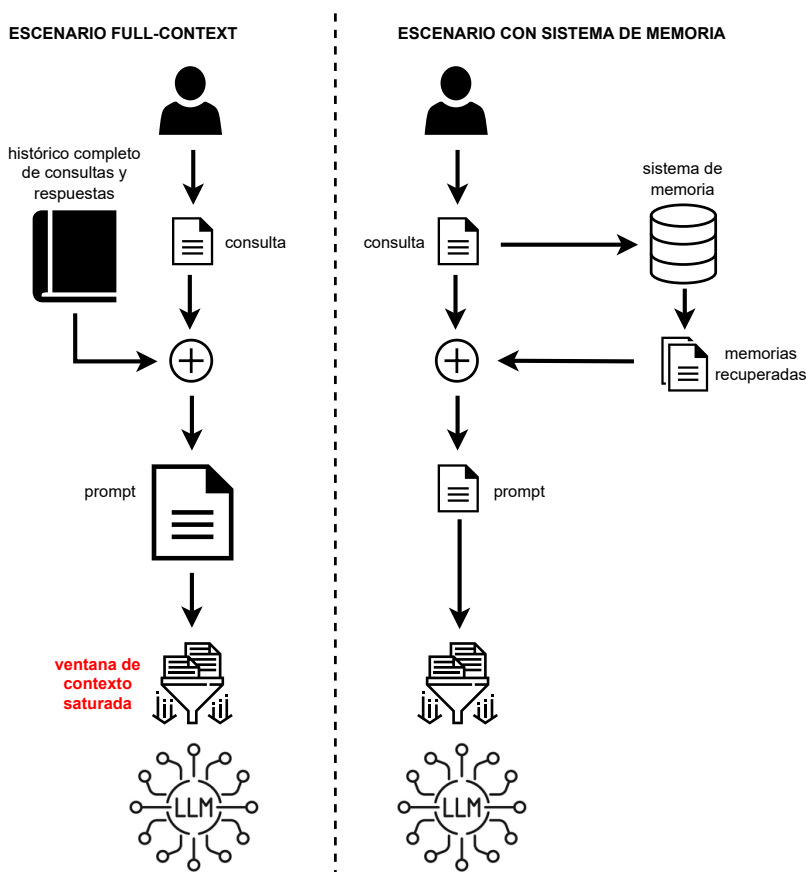


Figura 2.4: Visualización de los dos escenarios de uso. Fuente: Elaboración propia.

para responder a la pregunta, que serán introducidas en el contexto del modelo para que este pueda responder a la pregunta utilizando únicamente esta información relevante.

En este escenario se evaluará entonces la capacidad del sistema de memoria para: 1º crear las memorias relevantes a lo largo de la conversación, 2º recuperar las memorias relevantes para responder a la pregunta y 3º mejorar el rendimiento del modelo al reducir la cantidad de información irrelevante que se introduce en el contexto.

2.3.2. Mem0

Entre los diferentes sistemas de memoria que hay en el mercado, Mem0 [2] ha sido elegido como base para este trabajo, permitiendo realizar la comparativa entre la ventana de contexto y el uso de un sistema de memoria. Esta decisión se ha tomado por ser Mem0 un ejemplo canónico de sistema de memoria, por su facilidad de uso e integración y por ser un sistema actual, con una versión *open source* continuamente actualizada.

Es por ende necesario entender el funcionamiento de Mem0 para poder comprender la relación entre la mejora que supone y los posibles factores negativos que se pueden asociar a este.

Mem0 es un sistema de memoria textual, pero su componente principal es una base de datos vectorial en la que se almacenarán embeddings de las memorias conforme se vayan generando, permitiendo una búsqueda y recuperación más eficiente. Mem0 actúa de wrapper sobre esta base de datos, permitiendo usar cualquier tipo de BBDD vectorial que se desee, lo que aporta una gran flexibilidad y, en caso de que sea necesario, privacidad.

Su comportamiento se divide en dos fases principales: la extracción de posibles memorias de la conversación, y la actualización de la base de datos:

La **extracción**, como se aprecia en la figura 2.5, se realiza en cada iteración del chat, tomando como entrada el último par de mensajes —usuario y respuesta del modelo— y una secuencia con los últimos m mensajes, siendo este número un hiperparámetro¹. Además, se utiliza un resumen de la conversación generado de manera asíncrona en la base de datos. Toda esta información se introduce en un *prompt* y se envía a un LLM, previamente configurado por el operador del sistema, con el fin de que este extraiga memorias candidatas para añadir a la base de datos.

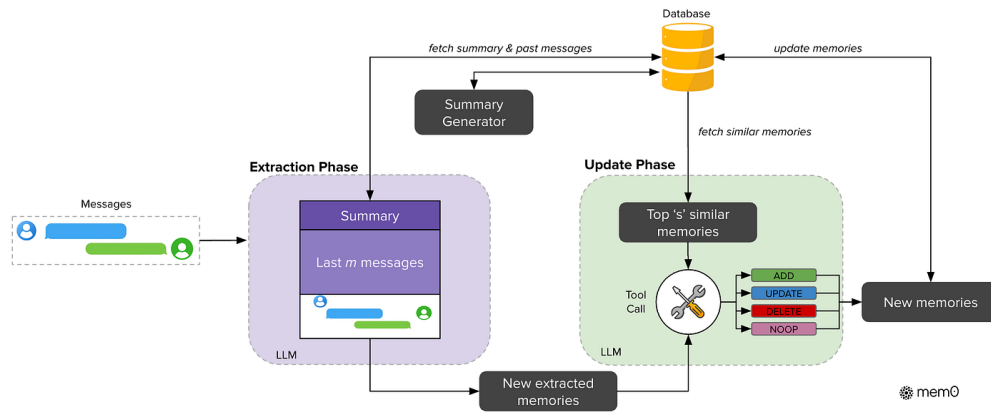


Figura 2.5: Pipeline de adición de memorias en Mem0. Extraído de [2].

Una vez realizada esta fase se procede con la de **actualización**. En esta se realiza el mismo procedimiento para cada memoria candidata extraída en el paso anterior, este consiste en realizar una búsqueda semántica en la base de datos tomando como prompt la memoria candidata. Entonces, usando tanto la memoria candidata como las memorias obtenidas en la consulta, se realiza una segunda llamada a un LLM, el cual debe decidir entre cuatro posibles acciones: *ADD*, *UPDATE*, *DELETE* y *NOOP*. Que significan:

- **NOOP**: No hacer nada, la memoria candidata no aporta nada nuevo a la base de datos.
- **UPDATE**: Actualizar alguna de las memorias obtenidas en la consulta con la información de la memoria candidata.
- **ADD**: Añadir la memoria candidata como una nueva memoria en la base de datos.

¹En [2], documento en el que se introduce este hiperparámetro, los autores indican que para sus experimentos establecen el valor por defecto en 10. En cuanto a la configuración para este proyecto, no se puede determinar con certeza, puesto que dicho parámetro no se encuentra expuesto para su modificación en la API actual ofrecida por Mem0.

- **DELETE:** Borrar alguna de las memorias obtenidas en la consulta, pues ya no es relevante.

En la figura 2.5 se puede observar la pipeline de creación de memorias de Mem0.

Finalmente, para la **recuperación** de memorias, Mem0 vuelve a hacer uso de un LLM. Este reescribe el mensaje del usuario y realiza una búsqueda semántica en la base de datos, devolviendo las memorias más relevantes para ser introducidas en el contexto del modelo (véase la figura 2.6).

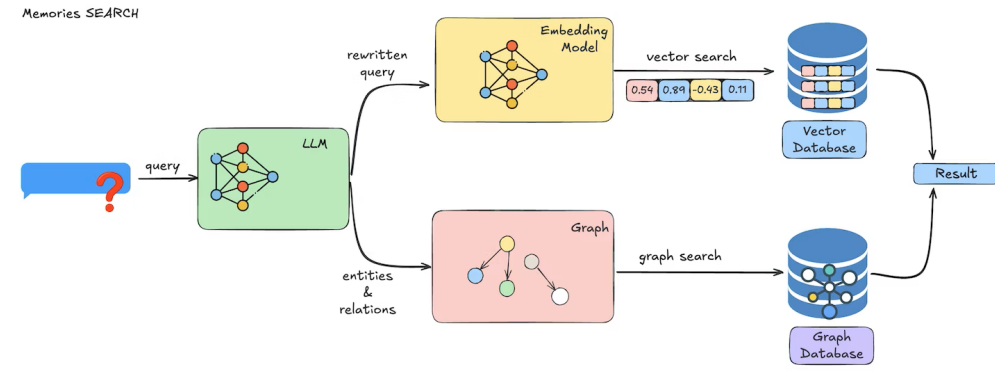


Figura 2.6: Pipeline de búsqueda de Mem0. Extraído de [2].

2.4. LongMemEval

Para evaluar el rendimiento que ofrece el uso de Mem0 frente al empleo exclusivo de la ventana de contexto, se ha elegido usar como benchmark el dataset LongMemEval [18] creado específicamente para evaluar las capacidades de memoria a largo plazo de los asistentes de IA.

El dataset se compone de un conjunto de 500 pares pregunta-respuesta, cada uno de ellos acompañado de un conjunto de sesiones de chat –una serie de interacciones o turnos usuario-asistente. En ellas se encuentra tanto la información necesaria para responder a la pregunta como conversación adicional irrelevante.

Formalmente, el conjunto de datos se define como $\{(q_i, a_i, \{S_i^j\})\}_{i=1}^{500}$, donde q_i representa la pregunta, a_i la respuesta correcta y $\{S_i^j\}$ el conjunto de sesiones asociadas recorridas por el índice j . En este contexto, la descomposición en turnos es $S_i^j = \{(u_k, r_k)\}$, donde u_k y r_k corresponden al mensaje del usuario y a la respuesta del asistente en el turno k , respectivamente.

LongMemEval cuenta con tres versiones distintas, diferenciadas por la cantidad de información irrelevante que se introduce en el contexto, lo que permite evaluar el rendimiento de los modelos en diferentes escenarios. Estas son:

- **LongMemEval-Oracle:** versión baseline que solo contiene sesiones de chat con información relevante para la pregunta.

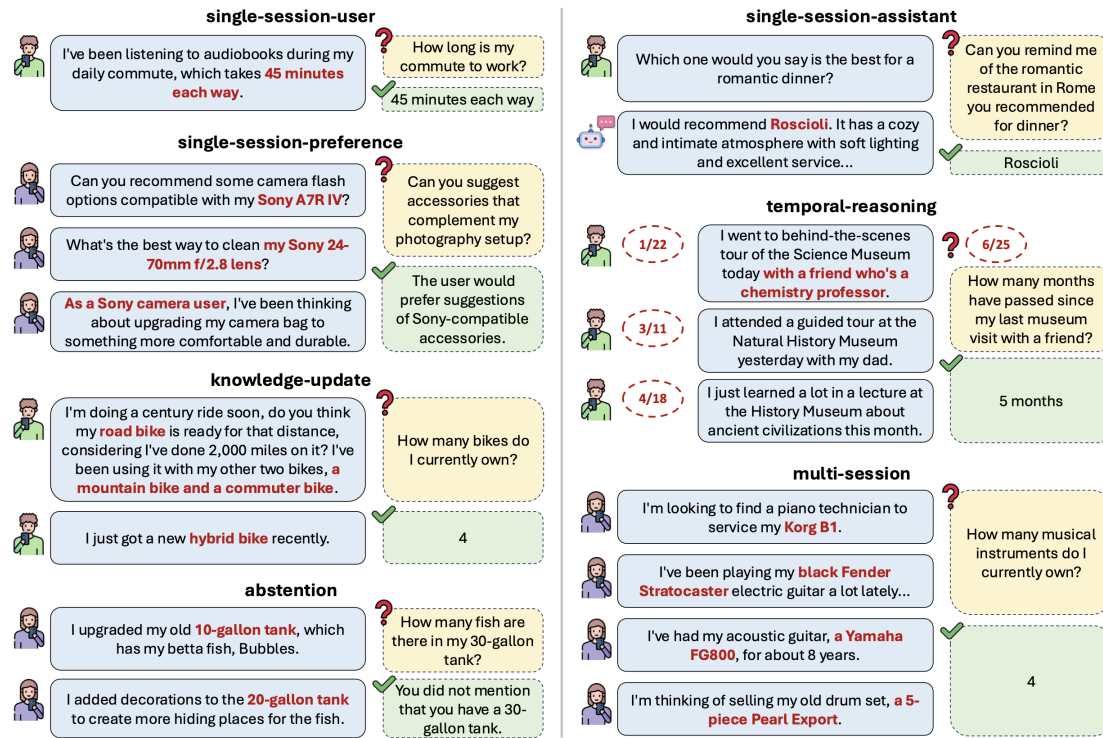


Figura 2.7: Ejemplos de los tipos de preguntas de LongMemEval. Extraído de [18].

- **LongMemEval-S:** versión que introduce una cantidad moderada de mensajes irrelevantes, con aproximadamente 115K tokens por pregunta.
- **LongMemEval-M:** versión extendida con una gran cantidad de mensajes irrelevantes, llegando al 1.5M de tokens para cada problema.

De ellas solo se han usado las dos primeras, pues la generación de memorias es un proceso lento y el número de tokens de la versión M hace que el coste computacional sea demasiado elevado, lo que no es viable para este trabajo.

En lo que corresponde a las preguntas en sí, estas se dividen en siete tipos distintos (véase la figura 2.7) y buscan evaluar cinco aspectos claves en el desarrollo de la memoria a largo plazo. Dichas habilidades y sus correspondientes tipos de preguntas son:

1. **Information Extraction:** la habilidad para extraer información relevante de una conversación larga. Las preguntas asociadas a esta habilidad son, por un lado *single-session-user* y *single-session-assistant*, que buscan evaluar la capacidad de memorizar información relevante de los mensajes del usuario o del asistente dentro de una sesión de chat; y por otro lado, *single-session-preference* donde se requiere memorizar o aprender preferencias del usuario.
2. **Multi-session Reasoning:** la habilidad para integrar información a lo largo de múltiples sesiones de chat y responder a preguntas complejas que requieren razonamiento inter-sesión. Las preguntas asociadas a esta habilidad son *multi-session* que evalúan la habilidad para agregar información a lo largo de múltiples sesiones de chat.

3. **Knowledge Updates:** la habilidad para reconocer cambios en la información y actualizar el conocimiento almacenado en consecuencia. Las preguntas asociadas a esta habilidad son *knowledge-update* que buscan evaluar la capacidad de actualizar el conocimiento almacenado en la memoria a largo plazo.
4. **Temporal Reasoning:** la habilidad para ser consciente de los aspectos temporales de la información. Sus preguntas asociadas son homónimas a la habilidad, es decir, *temporal-reasoning*, y buscan evaluar la capacidad de entender y razonar sobre la información temporal.
5. **Abstention:** la habilidad para identificar cuándo no se tiene suficiente información para responder a una pregunta y contestar indicándolo. Las preguntas asociadas a esta habilidad son *abstention*, y no son preguntas específicas sino que hay 30 preguntas diseminadas a lo largo de los otros seis tipos.

Cabe destacar que, aunque LongMemEval se ha diseñado para evaluar las capacidades de memoria a largo plazo de los asistentes de IA, no se ha creado específicamente para evaluar el rendimiento de los sistemas de memoria, por lo que no se ajusta perfectamente a este objetivo. En particular, el hecho de que para cada conjunto de sesiones haya una única pregunta asociada se aleja del uso real de los sistemas de memoria, donde se espera que a lo largo de una conversación se puedan realizar varias preguntas sobre la información almacenada en la memoria. Por ello, cuando se realice la comparativa entre escenarios de alguna de las métricas presentadas a continuación, se realiza un análisis adicional teniendo en cuenta la respuesta de N veces la misma pregunta sobre su contexto.

2.5. Métricas de evaluación

En esta sección se describen las diferentes métricas que se han usado para evaluar los distintos aspectos a tener en cuenta a la hora de comparar el uso exclusivo de la ventana de contexto frente al uso de un sistema de memoria. Estas métricas se han elegido para evaluar tanto el rendimiento del modelo a la hora de responder a las preguntas, como el coste computacional asociado a cada uno de los escenarios.

2.5.1. Métricas compartidas

Primero vamos a describir las métricas que se han evaluado en ambos escenarios:

- **Precisión:** métrica clave para evaluar el rendimiento del modelo a la hora de responder a las preguntas. Se calcula como el número de respuestas correctas dividido por el número total de preguntas, y se expresa como un porcentaje. Para poder determinar si una respuesta es correcta o no, se ha utilizado un modelo de lenguaje como juez, al que se le introduce la pregunta, la respuesta del modelo y la respuesta correcta proporcionada en el dataset.

- **Latencia:** mide el tiempo que tarda el modelo en generar una respuesta a una pregunta, desde el momento en que se introduce la pregunta hasta que se obtiene la respuesta. En el caso de los sistemas de memoria la latencia incluye el tiempo requerido para recuperar las memorias pero no el que se tarda en procesar la conversación para generar las memorias. Este tiempo se asume que ocurre de manera asíncrona durante la conversación, y no es un coste adicional a la hora de responder a la pregunta.
- **Tokens del prompt de entrada:** como su nombre indica esta métrica mide el número de tokens que tiene el prompt utilizado que se introduce al modelo para generar la respuesta a una pregunta. Esta métrica es importante pues el número de tokens del prompt afecta directamente al coste computacional de generar una respuesta, y a su vez, a la latencia.
- **Coste de CO₂:** esta métrica mide el impacto ambiental asociado al uso de los modelos de lenguaje y los sistemas de memoria, calculando la cantidad de dióxido de carbono emitida durante todo el proceso de generación de respuestas. Su cálculo específico se explica en su sección correspondiente, pero es importante destacar que esta métrica es crucial para evaluar la sostenibilidad de cada uno de los enfoques, especialmente teniendo en cuenta el aumento del uso de los modelos de lenguaje y su impacto ambiental asociado.

2.5.2. Métricas específicas del uso de un sistema de memoria

En el caso del uso de un sistema de memoria, se han evaluado además las siguientes métricas específicas:

- **Tiempo de creación de memorias:** esta métrica mide el tiempo que tarda el sistema de memoria en procesar la conversación y generar las memorias correspondientes. Este tiempo se asume que ocurre de manera asíncrona durante la conversación, y no es un coste adicional a la hora de responder a la pregunta, pero es importante medirlo para evaluar la eficiencia del sistema de memoria pues se usará junto a la latencia para calcular el coste de CO₂.
- **F1:** esta medida se calcula como $F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$ [20]. No obstante, la forma de calcular *precision* y *recall* depende de la unidad de análisis (tokens, memorias, sesiones o turnos). Esta métrica se usa ampliamente en la literatura, aunque su implementación varía según la tarea:
 - *Token-level F1:* en este enfoque, la *precision* y el *recall* se calculan a nivel de tokens, normalmente comparando la respuesta generada por el modelo con la respuesta de referencia del dataset. Aunque es una métrica común, puede no ser la más adecuada para evaluar sistemas de memoria, ya que se centra en la respuesta final y en el solapamiento léxico, pudiendo penalizar paráfrasis correctas [2].
 - *F1 con LLM juez:* en este trabajo se adopta un enfoque alternativo en el que se evalúa la calidad de las memorias recuperadas, no solo la respuesta final.

Para ello, un LLM recibe la pregunta, la respuesta de referencia y las memorias recuperadas, y clasifica cada memoria como relevante o no relevante (estimando así la *precision*). Además, el LLM estima la suficiencia del conjunto de memorias para responder la pregunta; a partir de esa suficiencia se aproxima el *recall* de forma heurística. Este enfoque es adecuado para la tarea, ya que mide directamente la utilidad de la memoria recuperada.

- *F1 con etiquetas del dataset*: este enfoque también evalúa las memorias recuperadas, pero usa anotaciones estructuradas del dataset en lugar de un LLM juez. En particular, permite medir la recuperación a nivel de sesión (*session-level*) y a nivel de turno exacto (*turn-level*), comprobando si las memorias provienen de las partes relevantes de la conversación. Es una medida complementaria a la anterior: aporta mayor objetividad estructural, aunque no evalúa de forma directa la relevancia semántica del contenido recuperado.

2.6. Cálculo del coste de CO₂

Como ya se ha indicado en la sección anterior, el coste de CO₂ es una métrica crucial a evaluar pues el impacto ambiental asociado a los sistemas de inteligencia artificial es significativo y está en constante crecimiento [5]. Sin embargo, no existe una metodología única estandarizada para realizar este cálculo.

En este trabajo se ha decidido partir de la metodología empleada por la calculadora MachineLearning Impact calculator descrita en [5], que estima las emisiones de carbono a partir del tiempo de ejecución, la potencia del hardware, la intensidad de carbono de la red eléctrica regional y eficiencia energética del datacenter.

El no utilizar directamente esta calculadora, sino solo su metodología, se debe a que esta asume una utilización constante del hardware al 100 % durante todo el proceso. Esta hipótesis no es adecuada para comparar los dos escenarios evaluados, pues tienen características muy distintas: por un lado, *FullContext* tiene una única inferencia con un *prompt* largo que incluye todo el contexto, y que requerirá mayor potencia. Por otro lado, *Mem0* tiene un primer flujo con múltiples llamadas al modelo para generar y actualizar memorias, donde cada llamada es mucho más corta y tiene un overhead adicional por la comunicación con la base de datos; y finalmente, una inferencia con memorias recuperadas, que trabaja con *prompts* mucho más cortos. Por ello, con el fin de aproximar estas diferencias, se introduce un factor de utilización U en la fórmula.

De esta manera, el coste de CO₂ equivalente se estima como:

$$CO_{2-eq} = t \cdot P_{GPU} \cdot EF \cdot I_{carbono} \cdot U \quad (2.1)$$

donde t es el tiempo de ejecución en horas, P_{GPU} la potencia del hardware expresada en kW (aproximada mediante el TDP), EF la eficiencia energética del datacenter, $I_{carbono}$ la intensidad de carbono de la región en gCO_{2-eq}/kWh, y U el factor de utilización durante el proceso.

En ausencia de telemetría detallada, se establecen valores heurísticos para U que reflejan de forma aproximada cómo cada escenario utiliza el hardware. Para el escenario *FullContext*, que mantiene el hardware procesando continuamente un *prompt* largo en una única llamada, se asume $U = 1.0$. Para el proceso de creación y actualización de memorias, que implica múltiples llamadas al modelo de menor tamaño cada una, con overhead adicional de acceso a la base de datos, se estima $U = 0.5$. Finalmente, para la inferencia con memorias recuperadas, que trabaja con *prompts* mucho más cortos, se asume $U = 0.2$.

Para completar el resto de parámetros se adoptan los siguientes valores: $P_{GPU} = 0.7$ kW (700 W), coherente con el hardware utilizado (véase 3.2.2); $EF = 1.5$, como valor típico para datacenters privados [6]; e $I_{carbono} = 100 \text{ gCO}_{2eq}/kWh$ como valor estimado según los datos presentados por Red Eléctrica de España en [14]. Estos valores, en conjunción con el factor de utilización, permiten dar una estimación realista y objetiva del coste de CO2 asociado a cada uno de los escenarios evaluados, teniendo en cuenta sus características específicas de uso del hardware.

Terminamos esta sección presentando la comparativa del coste de responder N veces una pregunta sobre el mismo contexto para cada uno de los escenarios, que se modela de la siguiente manera:

$$CO2_{eq}^{FullContext} = N \cdot CO2_{eq}(Inferencia_{FullContext}) \quad (2.2)$$

$$CO2_{eq}^{Mem0} = CO2_{eq}(Creacion\ de\ memorias) + N \cdot CO2_{eq}(Inferencia_{Mem0}) \quad (2.3)$$

En el caso de Mem0, el primer término representa un coste fijo de construcción de memorias que no depende de N , mientras que solo el segundo término escala con el número de consultas. Esta descomposición permite analizar en qué condiciones el coste inicial de crear memorias puede amortizarse frente a repetir múltiples inferencias con contexto completo.

Análisis de objetivos y metodología

Una vez tratado el estado del arte y presentados los conceptos teóricos que enmarcan a los LLMs y los sistemas de memoria, realizamos en este capítulo el análisis de los objetivos del mismo, estableciendo los elementos concretos que nos van a permitir llevar a cabo dichos objetivos así como la metodología de desarrollo.

3.1. Objetivos

El objetivo principal de este trabajo es analizar el impacto que tienen los sistemas de memoria en el rendimiento de los grandes modelos de lenguaje (LLMs) mediante la creación de un pipeline de evaluación. Con el fin de alcanzar este objetivo, se han establecido los siguientes subobjetivos:

- **Entender los sistemas de memoria:** Realizar un estudio teórico detallado sobre los sistemas de memoria en LLMs, examinando y comprendiendo la literatura actual sobre el tema y consultando implementaciones concretas.
- **Hacer uso de Mem0:** Entender y utilizar un sistema de memoria específico para integrar las funcionalidades de memoria en los LLMs dentro del flujo de trabajo.
- **Crear un entorno de evaluación:** Desarrollar un pipeline de evaluación que ensamble las herramientas necesarias para permitir medir el impacto de los sistemas de memoria en el rendimiento de los LLMs.
- **Establecer métricas de rendimiento:** Definir y establecer métricas específicas para evaluar el rendimiento de los LLMs con y sin la integración de sistemas de memoria, asegurando que estas métricas sean relevantes y adecuadas para el análisis.
- **Evaluar el rendimiento:** Realizar una evaluación exhaustiva del rendimiento de los LLMs con y sin la integración de sistemas de memoria, utilizando métricas específicas y el dataset LongMemEval-S.

- **Evaluar el impacto ambiental:** Analizar el impacto ambiental de la utilización de sistemas de memoria en los LLMs, considerando el consumo de recursos y la huella de carbono asociada.
- **Extraer conclusiones y plantear futuras líneas de investigación:** Analizar los resultados obtenidos en la evaluación para extraer conclusiones sobre el impacto de los sistemas de memoria en el rendimiento de los LLMs, y proponer posibles líneas de investigación futura basadas en los hallazgos obtenidos.

3.2. Herramientas y tecnologías

Para el desarrollo de este proyecto se ha hecho uso de un amplio abanico de herramientas y tecnologías, que han hecho posible la implementación y desarrollo de los objetivos planteados. A continuación se detallan las principales herramientas utilizadas:

3.2.1. Entorno de Programación

Se ha trabajado usando **Python** como lenguaje de programación principal, debido a la amplia disponibilidad de librerías que han permitido la implementación de los diferentes componentes del sistema. Todas estas librerías han sido gestionadas dentro de un entorno virtual creado con **venv**, lo que ha permitido mantener un entorno de desarrollo limpio y organizado. Entre ellas, destacan:

- **Mem0:** librería oficial de Mem0, permitiendo una integración sencilla de sus funcionalidades tanto en local como a través de su API. En particular, se ha hecho uso de la versión 1.0.1.
- **Ollama:** librería oficial de Ollama, que ha permitido la utilización de diferentes modelos de lenguaje en el sistema desarrollado.
- **OpenAI:** librería oficial de OpenAI, que ha sido fundamental para la integración de los modelos de lenguaje de OpenAI en el sistema.
- **tiktoken:** librería de OpenAI para el conteo de tokens, necesaria para la medición de la cantidad de tokens utilizados en las diferentes interacciones con los modelos de lenguaje.
- **Qdrant:** librería oficial de Qdrant, que ha permitido la integración de este sistema de base de datos vectorial en el proyecto.

Como entorno de desarrollo integrado (IDE), se ha utilizado **Visual Studio Code**, que ofrece una gran cantidad de extensiones y herramientas para facilitar el desarrollo en Python. Además, permite una integración fluida con entornos remotos a través de la extensión **Remote - SSH**, lo que ha sido fundamental para trabajar con la máquina proporcionada por la universidad.

3.2.2. Entorno de desarrollo

Para el desarrollo de este proyecto se han utilizado, de manera directa, tres máquinas diferentes.

En primer lugar, para el desarrollo de código y las pruebas iniciales se usó un **ordenador personal** con las siguientes características: procesador Intel Core i7-11370H de 11ª generación, 16 GB de RAM, y una tarjeta gráfica NVIDIA GeForce RTX 3060 Laptop, todo bajo un sistema operativo Windows 11. Este ordenador fue clave para el desarrollo inicial del proyecto, permitiendo la implementación y prueba con pequeños modelos de lenguaje.

Posteriormente, con el fin de poder trabajar con modelos de lenguaje más grandes y realizar pruebas más exhaustivas, se hizo uso de **Ollama en un servidor del grupo de investigación Sistemas Inteligentes y Telemática de la Universidad de Murcia**. Dicho servidor tiene las siguientes características: sistema operativo Debian GNU/Linux 12 (bookworm), procesador AMD EPYC 9554 a 3.095 GHz, dos GPUs NVIDIA GH100 y 515975 MiB de memoria RAM (aprox. 516 GB). Esta máquina permitió la ejecución de modelos de lenguaje más complejos y la realización de pruebas a mayor escala.

Además, para poder alojar la base de datos vectorial empleada para Mem0, y poder ejecutar pruebas largas sin necesidad de mantener el ordenador personal encendido, se utilizó una **máquina virtual proporcionada por el mismo grupo de investigación**. Esta máquina virtual tenía las siguientes características: Debian GNU/Linux 13 (trixie), 4 vCPU Intel Xeon Silver 4214R a 2.40 GHz, 3.8 GiB de RAM y 60 GB de almacenamiento.

En esta última máquina virtual se alojó la base de datos vectorial de **Qdrant**, herramienta de código abierto que ha permitido la gestión de la base de datos vectorial utilizada para almacenar los vectores de memoria generados por Mem0.

3.2.3. Modelos de lenguaje

Para el desarrollo de este proyecto se han utilizado diferentes modelos de lenguaje, tanto locales como a través de API, con el fin de evaluar su rendimiento y capacidad de integración con el sistema desarrollado. Para esto se ha hecho uso de la herramienta de código abierto **Ollama**, que permite la ejecución de diferentes modelos de lenguaje en local, y de la API de **OpenAI**, que ha permitido la integración de los modelos de lenguaje de OpenAI en el sistema desarrollado.

Los modelos de lenguaje utilizados en este proyecto han sido los siguientes:

- **llama3.3:latest**: Modelo de 70b de parámetros, desarrollado por Meta. Es un modelo con un gran rendimiento y, por tanto, ha sido usado como modelo de referencia para las pruebas, se ha usado como modelo local para Mem0 y ha sido, a través de todo el proyecto, el modelo evaluador de la *precisión* de los modelos de lenguaje.
- **qwen2.5:72b**: Modelo de 72b de parámetros, desarrollado por Alibaba. Modelo con un mayor número de parámetros que llama3.3, para poder evaluar el impacto de

un modelo más grande en el rendimiento del sistema desarrollado.

- **qwen3:32b**: Modelo de 32b de parámetros, desarrollado por Alibaba. Modelo con un menor número de parámetros que llama3.3, para poder evaluar el impacto de un modelo más pequeño en el rendimiento del sistema desarrollado.
- **mistral:8x22b**: Este modelo se eligió a posteriori para evaluar la pérdida de rendimiento ante un exceso en la ventana de contexto. Mientras que los modelos qwen solo permiten una entrada de 32k tokens, demasiado pequeña para la entrada de LongMemEval-S; y llama3.3 permite entradas de hasta 128k tokens, cabiendo perfectamente cada pregunta de LongMemEval-S; este modelo permite entradas de hasta 64k tokens, lo que lo hace ideal para evaluar el impacto de un contexto superior a la capacidad de la ventana.
- **GPT-5 nano**: Los modelos GPT-5 son los últimos modelos desarrollados por OpenAI a fecha de la redacción de este proyecto, y entre ellos se ha elegido el modelo *nano* por ser el más eficiente en cuestión de precio-rendimiento. Introducir un modelo de OpenAI permite evaluar el rendimiento de nuestro sistema frente al estándar de facto de la industria, permitiendo así una comparativa más completa y relevante.

Además de estos LLMs, también se ha hecho uso de diferentes modelos de *embedding*, pequeños modelos de lenguaje especializados únicamente en la generación de vectores de *embedding* para almacenar en la base de datos vectorial de Qdrant. En concreto, se han utilizado los siguientes modelos de *embedding*:

- **nomic-embed-text**: Modelo de embedding de 768 dimensiones, desarrollado por Nomic, y que se ha utilizado como estándar de referencia para la generación de los vectores de embedding en el sistema desarrollado.
- **qwen3-embedding:0.6b**: Modelo de embedding de 1024 dimensiones, desarrollado por Alibaba, y que se ha utilizado para evaluar el impacto de modelos de embedding más grandes en el rendimiento del sistema desarrollado.
- **text-embedding-3-small**: Modelo de *embedding* de 1536 dimensiones, desarrollado por OpenAI, se ha utilizado en conjunto con el modelo GPT-5 nano para evaluar el rendimiento de un sistema completamente basado en OpenAI.

3.3. Metodología de trabajo

Para el desarrollo correcto del trabajo se han ido realizando una serie de reuniones periódicas con el tutor. Estas reuniones, llevadas a cabo semanalmente, han permitido ir marcando los objetivos a corto plazo y asegurar que el desarrollo del proyecto se mantenga dentro de los objetivos planteados. A continuación se detalla, de manera general y por quincenas, el desarrollo del proyecto:

- **15/09/2025 - 28/09/2025**: Durante esta primera quincena se llevaron a cabo las reuniones iniciales para establecer el foco y los objetivos del proyecto, estableciendo

Mem0 como el sistema base de memoria a utilizar. Además, se revisaron los papers fundamentales a seguir para el desarrollo del proyecto, incluyendo los de Mem0 [2] y LongMemEval [18].

- **29/09/2025 - 12/10/2025:** Durante esta segunda quincena se llevó a cabo la instalación inicial de las herramientas del proyecto, haciendo pruebas aisladas de los diferentes sistemas: Ollama, Mem0, Qdrant, etc. Además, se comenzó a trabajar en la integración de Mem0 con Ollama, para poder usar los modelos de lenguaje locales con el sistema de memoria.
- **13/10/2025 - 26/10/2025:** Esta tercera quincena fue la más productiva en cuanto al desarrollo del código del proyecto, ya que se construyeron todas las abstracciones necesarias para el desarrollo de la pipeline de evaluación, y se hizo una implementación inicial de estas con los sistemas previamente mencionados.
- **27/10/2025 - 09/11/2025:** En esta cuarta quincena se instaló y configuró la base de datos vectorial de Qdrant en la máquina virtual, y se integró esta base de datos con el sistema de memoria de Mem0. Además, se creó un evaluador de *precisión* usando llama3.3, y se hicieron las primeras pruebas de rendimiento con el dataset LongMemEval-Oracle.
- **10/11/2025 - 23/11/2025:** Durante esta quinta quincena se llevaron a cabo las pruebas de rendimiento con el dataset LongMemEval-S, usando los diferentes modelos de lenguaje mencionados anteriormente. Además, se comenzaron a analizar los resultados obtenidos en estas pruebas.
- **24/11/2025 - 18/1/2026:** Durante el mes de diciembre y parte de enero se hizo un parón en el desarrollo debido a los exámenes y las vacaciones de Navidad, retomando el proyecto a finales de enero.
- **19/1/2026 - 1/2/2026:** Durante esta sexta quincena se retomó el proyecto, se añadieron nuevas métricas de rendimiento a la pipeline, se realizaron pruebas usando ya el dataset LongMemEval-S y se comenzó a escribir el informe del proyecto.

Diseño y resolución

Este capítulo inicia, en la sección 4.1, con una breve descripción del diseño del pipeline de evaluación creado para evaluar los diferentes escenarios de uso presentados. Posteriormente, en las secciones 4.2 y 4.3 se presentan los resultados de la evaluación realizada con el dataset LongMemEval, tanto para su versión Oracle como para su versión S. Finalmente, en la sección 4.4, se exploran diferentes líneas de mejora del sistema de memoria Mem0 a partir de los resultados obtenidos.

4.1. Pipeline de evaluación

Con el fin de crear un entorno de evaluación lo más extensible posible, se decidió crear un pipeline de procesamiento de datos modular, de manera que las diferentes partes que lo componen puedan ser fácilmente intercambiables (véase la figura 4.1).

Para ello, se crearon una serie de clases abstractas que definen la interfaz de cada una de las partes del pipeline, y una clase principal *Pipeline*, que se encarga de gestionar el flujo de datos entre dichas partes. Las clases principales son las siguientes:

- La clase *DatasetReader*, que se encarga de acceder a las preguntas del dataset, adaptándose a su estructura concreta, y las devuelve en un formato común establecido por la clase *Pregunta*.
- La clase *MemorySystem*, que representa la lógica de procesamiento de las preguntas, iterando sobre los objetos de tipo *Pregunta* generados en el paso anterior y aplicando dos funciones: una de ellas se encarga de procesar el contexto de la pregunta y la otra de responder a la pregunta utilizando dicho procesado, devolviendo un objeto de tipo *Respuesta*. Aunque la clase se llame *MemorySystem*, heredando de ella se han implementado los dos escenarios vistos en la sección 2.3.1, es decir, tanto el escenario de memoria como el de contexto completo.

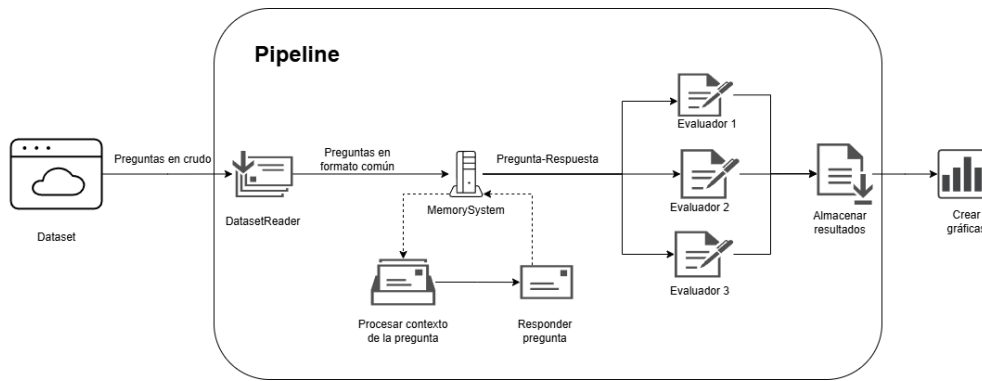


Figura 4.1: Esquema del pipeline de evaluación diseñado para este proyecto.

- La clase *Evaluator*, encargada de evaluar alguna de las métricas explicadas en la sección 2.5, recibe tanto los objetos *Pregunta* como *Respuesta* y utiliza la información de ambos para calcular la métrica correspondiente. Aunque aquí se hable de la clase *Evaluator*, lo normal es que se creen varias clases que hereden de esta, cada una de ellas encargada de calcular una métrica concreta, y el pipeline se encarga de ejecutarlas todas.

Cabe destacar que tanto la clase *MemorySystem* como la clase *Evaluator* pueden hacer uso de modelos de lenguaje para realizar su función, por lo que se ha creado una clase abstracta *LLMBackend* que define la interfaz de estos modelos, permitiendo trabajar de manera uniforme con diferentes API del mercado, como OpenAI, Ollama, etc.

Finalmente, el pipeline recibe la salida de los diferentes evaluadores y escribe los resultados de la evaluación en un archivo JSON. Este formato de salida permite una visualización sencilla de los resultados y facilita la comparación entre diferentes configuraciones del sistema o distintos modelos de lenguaje. Además, su estructura permite una integración sencilla con scripts de generación de gráficos o tablas para la presentación de resultados.

4.1.1. Implementación concreta

Partiendo de esta base, se han implementado diferentes clases concretas para poder realizar la evaluación de los diferentes escenarios de uso presentados en la sección 2.3.1 y de las diferentes métricas explicadas en la sección 2.5. En particular, se han implementado las siguientes clases concretas:

- *DatasetReader*:
 1. *LongMemEvalReader*, el único dataset utilizado en este proyecto es el LongMemEval (véase la sección 2.4), por lo que solo se ha implementado un lector específico para este dataset, que se encarga de acceder a las preguntas y respuestas del mismo y adaptarlas al formato común establecido por la clase *Pregunta*.

■ *MemorySystem*:

1. *FullContext*, clase simple que implementa el escenario de contexto completo, es decir, no realiza ningún tipo de procesamiento sobre el contexto de la pregunta, sino que simplemente lo pasa al modelo de lenguaje tal cual.
2. *Mem0_local*, clase que implementa el escenario de memoria, utilizando el sistema de memoria Mem0 (véase la sección 2.3.2), en particular su versión local, es decir, donde los datos de memoria se almacenan localmente.
3. *Mem0_api*, clase que implementa el escenario de memoria, utilizando el sistema de memoria Mem0, en particular su versión API, es decir, donde los datos de memoria se almacenan a través de una API externa.

■ *Evaluator*:

1. *LLMJudgeEvaluator*, evaluador principal de exactitud basado en un modelo de lenguaje como juez. Compara respuesta generada y respuesta objetivo con una plantilla adaptada al tipo de pregunta, y decide si la respuesta es correcta.
2. *F1ScoreEvaluator*, evaluador léxico que calcula precisión, *recall* y F1 a nivel de token entre la respuesta predicha y la respuesta de referencia.
3. *LatencyEvaluator*, evaluador de rendimiento temporal durante la generación de respuesta, que reporta estadísticas agregadas como media, mediana y percentiles.
4. *ContextProcessingTimeEvaluator*, evaluador del tiempo invertido en procesar el contexto antes de responder, útil para comparar el coste de preparación entre sistemas.
5. *ReferenceAccuracyEvaluator*, evaluador orientado a la calidad de recuperación de memoria. Usa un juez LLM para estimar relevancia de memorias recuperadas y suficiencia del conjunto recuperado, y a partir de ello calcula precisión, *recall* y F1.
6. *SessionPrecisionEvaluator*, evaluador de recuperación a nivel de sesión; mide si las memorias recuperadas provienen de las sesiones correctas respecto a las sesiones que contienen la respuesta.
7. *TurnPrecisionEvaluator*, evaluador de recuperación a nivel de turno; mide si las memorias recuperadas corresponden a los turnos exactos donde está la información de respuesta.
8. *MemoryTokenUsageEvaluator*, evaluador de eficiencia en uso de contexto, que cuantifica los tokens del *prompt* usado para responder y resume el consumo total y por tipo de pregunta.
9. *MemoryAuditEvaluator*, evaluador de auditoría e inspección manual; para cada pregunta muestra las memorias recuperadas frente al conjunto total almacenado, facilitando análisis cualitativo del comportamiento del sistema de memoria.

4.2. Evaluación con LongMemEval-Oracle

La primera evaluación se ha realizado con la versión Oracle de LongMemEval. Esta variante es la más ligera, ya que, como se explicó en el estado del arte (véase sección 2.4), el contexto de cada pregunta se restringe a las sesiones relevantes para responderla. Con ello se evalúa el rendimiento de ambos escenarios en condiciones cercanas a las ideales, al minimizarse el ruido en el contexto.

En la Tabla 4.1 se resume estadísticamente el conjunto de conversaciones evaluado. Los datos corroboran lo anterior: el contexto por pregunta se limita a pocas sesiones, lo que se refleja en un número moderado de turnos y tokens. Aun así, hay variabilidad entre preguntas, lo que permite evaluar el rendimiento en condiciones diversas.

Métrica	Min	Max	Media	Mediana
Sesiones	1	6	1,90	2
Turnos	2	72	21,92	24
Tokens	110	22.098	5.639,21	5.661

Tabla 4.1: Resumen estadístico del contexto por pregunta en LongMemEval-Oracle.

En este dataset es razonable esperar un rendimiento alto del escenario de contexto completo, puesto que el contexto de cada pregunta, no es extenso, se ha construido para ser suficientemente informativo y, además, evita información irrelevante que pudiera degradar la respuesta. Por su parte, el escenario de memoria también debería obtener buenos resultados, aunque podría no igualar al escenario de contexto completo en este entorno “limpio”, puesto que la generación de memorias conlleva de manera inherente una síntesis de la información en la que se pueden perder detalles importantes para la respuesta.

Durante el resto de la sección se presentan los resultados de la evaluación para los dos escenarios considerados. En el escenario de contexto completo (FullContext) se han ejecutado tres modelos distintos: Llama3.3, Mistral y un modelo de OpenAI (GPT-5 nano). Dado que este enfoque depende fuertemente del modelo base, disponer de variedad resulta informativo (véase la sección 3.2.3). Por otro lado, para el escenario de memoria (Mem0) del que se presentan los resultados se ha usado únicamente Llama3.3, tanto para generar las memorias como para responder, puesto que el objetivo aquí es presentar una comparativa de los escenarios de uso.

En la Tabla 4.2 se compara la **precisión** estimada mediante un evaluador LLM-Judge. Además del resultado global, se muestran los valores desglosados por tipo de pregunta, lo que permite un análisis más detallado.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	53,8 %	65,0 %	72,6 %	52,2 %
temporal-reasoning	24,8 %	35,3 %	51,1 %	36,8 %
multi-session	27,8 %	63,9 %	75,2 %	53,4 %
knowledge-update	75,6 %	70,5 %	70,5 %	69,2 %
single-session-preference	56,7 %	50,0 %	60,0 %	66,7 %
single-session-assistant	100,0 %	98,2 %	100,0 %	14,3 %
single-session-user	95,7 %	97,1 %	94,3 %	84,3 %

Tabla 4.2: Comparativa de la precisión usando LLM-Judge.

Atendiendo únicamente al resultado global destacan dos puntos importantes: Primero, dentro del escenario de contexto completo se observa que un modelo más potente mejora el rendimiento de forma apreciable (del orden del 10 % entre modelos). Segundo, el escenario con Mem0 presenta el valor más bajo (52,2 %). Por lo que, si se considera solo esta métrica y bajo las condiciones de LongMemEval-Oracle, el escenario de contexto completo, especialmente con OpenAI, parece la mejor opción.

No obstante, el desglose por tipo de pregunta permite interpretar mejor el comportamiento observado. En el escenario de contexto completo, la mayor degradación aparece en *temporal-reasoning*, que exige razonar sobre la secuencia temporal de eventos y sus relaciones. En el escenario de memoria, el rendimiento es relativamente homogéneo en la mayoría de categorías; sin embargo, *single-session-assistant* cae de forma marcada, lo que explica buena parte del descenso global. Una explicación plausible es que Mem0 tiende a no almacenar con la misma fidelidad información contenida en respuestas del asistente, precisamente la que este tipo de preguntas requiere.

En síntesis, para contextos relativamente pequeños y depurados, el escenario de contexto completo ofrece la mayor precisión, al proporcionar al modelo toda la información necesaria sin pérdida por recuperación. Aun así, salvo en preguntas de tipo *single-session-assistant*, el escenario de memoria se presenta como una alternativa razonable: aunque su precisión es inferior, no resulta inmediatamente descartable y puede compensar cuando se tienen en cuenta el resto de métricas que se analizan a continuación.

La primera de estas métricas a comparar es la **latencia**, entendida como el tiempo transcurrido desde que se recibe la pregunta hasta que se genera la respuesta. Esta métrica es especialmente relevante en sistemas en tiempo real y es donde el escenario de memoria comienza a mostrar ventajas. Además, es razonable esperar que dicha ventaja aumente en la siguiente versión del dataset, al crecer el contexto por pregunta pues el escenario de contexto completo deberá procesar más tokens en cada consulta, mientras que el escenario de memoria recuperará solo la información relevante manteniendo el tiempo de respuesta.

En las Tablas 4.3 y 4.4 se muestran los resultados, para la media y la mediana de la latencia respectivamente. En todos los casos la media es ligeramente superior, previsiblemente por la composición del dataset observada y por la sensibilidad de la media a los valores extremos, por lo que el análisis se apoya principalmente en la mediana.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	7,597	8,064	5,562	1,641
temporal-reasoning	12,000	9,730	5,991	1,924
multi-session	8,271	12,000	6,417	1,953
knowledge-update	6,087	7,871	5,025	1,125
single-session-preference	7,367	6,177	13,000	2,952
single-session-assistant	2,097	2,135	3,180	1,026
single-session-user	3,253	3,650	2,405	1,014

Tabla 4.3: Latencia(segundos) media en LongMemEval-Oracle.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	5,808	7,657	3,960	1,139
temporal-reasoning	6,199	8,769	4,416	1,169
multi-session	7,717	9,403	4,525	1,717
knowledge-update	5,996	7,673	4,302	0,988
single-session-preference	6,627	5,775	13,000	2,605
single-session-assistant	1,879	2,040	2,776	0,848
single-session-user	3,292	3,686	2,049	0,933

Tabla 4.4: Latencia(segundos) mediana en LongMemEval-Oracle.

En lo que respecta al resultado global, el escenario de memoria presenta menor latencia que todos los escenarios de contexto completo, siendo 3,48 veces más rápido que el escenario de contexto completo con el modelo de OpenAI (el más rápido dentro de FullContext). En consecuencia, bajo LongMemEval-Oracle, el escenario de memoria es el que ofrece mejor rendimiento en latencia.

Analizando por tipo de pregunta, se observan patrones similares en FullContext con Llama3.3 y Mistral: salvo *single-session-assistant* y *single-session-user*, que tienden a resolverse algo más rápido, el resto de categorías presentan latencias parecidas. En cambio, el escenario de OpenAI y el de Mem0 muestran una distribución más uniforme y baja, con la excepción de *single-session-preference*, cuya latencia es notablemente superior en ambos casos. Este comportamiento sugiere que estas preguntas requieren mayor esfuerzo de razonamiento; cuando el modelo base es más capaz (OpenAI) o el contexto recuperado es más focalizado (Mem0), ese esfuerzo adicional puede manifestarse como un incremento del tiempo de generación.

Aun así, el desglose por tipo, a diferencia de como pasaba con la precisión, no introduce hallazgos cualitativamente distintos del resultado global: Mem0 mantiene una ventaja consistente en latencia. Es razonable anticipar que dicha ventaja se mantenga, e incluso aumente, en la siguiente versión del dataset, donde el contexto de cada pregunta es mayor.

La segunda métrica analizada es el **número de tokens del prompt de entrada**, que cuantifica el número de tokens que se introducen al modelo para que esta responda cada pregunta. Esta métrica evalúa la eficiencia en el uso del contexto y es donde el escenario de memoria muestra una ventaja especialmente marcada, al recuperar únicamente la información relevante.

A diferencia de la métrica anterior, se presenta una única tabla con resultados globales de media y mediana (Tabla 4.5). En este caso se muestra un único modelo para el escenario de contexto completo (Llama3.3), ya que el *prompt* sigue una plantilla fija en la que se inserta la pregunta y el contexto sin procesarlo; por tanto, el número de tokens de contexto es esencialmente el mismo para todos los modelos en FullContext.

En esta métrica se observa una diferencia clara entre escenarios: la media de tokens de contexto en el escenario de contexto completo es 5786, mientras que en memoria es 101. Esto implica que Mem0 utiliza un 98,25 % menos de tokens de contexto para responder.

No obstante, esta cifra debe interpretarse con cautela pues aunque en la inferencia se utilicen pocos tokens, Mem0 necesita procesar el contexto entero para generar las memorias, lo que introduce un coste adicional de tokens no medidos.

En consecuencia, y como se expone de manera numérica más adelante, la ventaja de Mem0 se observa cuando se formulan varias preguntas sobre un mismo contexto, ya que el coste de procesamiento se amortiza entre consultas; mientras que en FullContext, cada pregunta vuelve a incorporar y procesar el contexto completo.

Métrica	FullContext-Llama3.3	Mem0-Llama3.3
Media	5 786	101
Mediana	5 806	100

Tabla 4.5: Uso global de tokens para la inferencia en LongMemEval-Oracle.

La última métrica comparativa entre ambos escenarios es el coste de CO₂; sin embargo, para realizar dicho cálculo es necesario presentar antes el **tiempo de procesamiento del contexto** en el escenario Mem0, es decir, el tiempo invertido en procesar la conversación para extraer y consolidar las memorias. Este coste no está incluido en la latencia mencionada anteriormente (que considera el tiempo de respuesta una vez que el sistema ya dispone de memorias), pero sí es necesario para estimar el coste energético total del enfoque y realizar una comparativa honesta frente al escenario FullContext.

Como se observa en la Tabla 4.6, el tiempo de procesamiento del contexto presenta una mediana considerable (212 s), que supera ampliamente a la mediana de la latencia de inferencia en FullContext (en torno a 8 s en el peor caso de los evaluados). Esto puede llevar a pensar que el escenario de memoria es considerablemente más lento. Sin embargo, hay que tener en cuenta que este proceso se realiza en segundo plano, de manera asíncrona, mientras que el usuario mantiene la conversación con el sistema.

Métrica	Valor (s)
Media	230
Mediana	212

Tabla 4.6: Tiempo de procesamiento del contexto en Mem0 (LongMemEval-Oracle).

Finalizamos entonces esta comparativa, analizando el **coste de CO₂** de ambos escenarios. En la tabla 4.7, se muestra la media del coste de CO_{2-eq} para cada escenario (salvo OpenAI al desconocerse su hardware), utilizando la metodología presentada en la sección 2.6.

En ella podemos ver que, el escenario de memoria presenta un coste de CO₂ menor en la inferencia, pero considerablemente más alto que el resto de escenarios cuando se le suma el coste de procesamiento del contexto.

Este fenómeno, que se anticipó ya cuando se analizó el número de tokens usados en la inferencia, se debe a que el proceso de generación de memorias de Mem0 es una inversión inicial, que busca amortizarse a largo plazo cuando se reiteren las preguntas sobre un mismo contexto.

En particular, en este caso el coste de CO₂ se amortiza a partir de la pregunta 15^a y 16^a, para Llama3.3 y Mistral respectivamente, lo que implica que, de media, si se formulan menos de 15 preguntas sobre un mismo contexto el escenario de contexto completo es más eficiente energéticamente, mientras que, a partir de ese punto, el escenario de memoria se vuelve más eficiente.

Métrica	FullContext-Mistral	FullContext-Llama3.3	Mem0-Llama3.3 (N=1)
Total (gCO ₂ -eq)	0.222	0.235	3.364
Inferencia (gCO ₂ -eq)	0.222	0.235	0.0096
Generación (gCO ₂ -eq)	-	-	3.354

Tabla 4.7: Comparativa del coste de CO₂ por consulta en LongMemEval-Oracle.

En conclusión, bajo las condiciones de esta versión del dataset, el escenario de contexto completo ofrece una mayor precisión y un coste de CO₂ menor para un número reducido de preguntas sobre un mismo contexto, mientras que el escenario de memoria presenta una latencia significativamente menor y se vuelve más eficiente energéticamente a medida que se formulan más preguntas sobre el mismo contexto, amortizando así el coste inicial de generación de memorias.

Una vez analizados los resultados comparativos, queda el estudio de la métrica **F1** sobre el escenario de memoria. En la Tabla 4.8 se muestran los resultados globales de esta métrica y sus componentes (véase la Sección 2.5). En ella se aprecia que el valor de la exhaustividad (*recall*) es alto, pero el de la precisión es bajo, por lo que el valor final del F1 también lo es. Esto indica que Mem0 suele recuperar memorias relacionadas con la pregunta, pero también incorpora información irrelevante; de este modo, la precisión

es el factor que más penaliza el valor final. En otras palabras, en este escenario limpio, el sistema recupera memorias coherentes, pero extrae contexto en exceso.

Método	F1	Precisión	Recall
LLM Juez	27.6	20.3	62.3

Tabla 4.8: Comparativa del F1 para el escenario de memoria en LongMemEval-Oracle.

4.3. Evaluación con LongMemEval-S

Tras analizar el caso base (LongMemEval-Oracle), resulta necesario estudiar cómo escalan ambos escenarios cuando el contexto por pregunta crece y se vuelve más ruidoso, aproximándose a condiciones de uso más realistas. Para ello se utiliza LongMemEval-S (véase la sección 2.4), que amplía el historial de cada pregunta con sesiones adicionales no relevantes. Esta ampliación incrementa de forma sustancial el número de tokens a procesar y diluye la información útil, convirtiendo esta variante en una prueba más exigente.

En la Tabla 4.9 se presentan estadísticas descriptivas del contexto por pregunta en LongMemEval-S. En comparación con la versión Oracle, hay un aumento marcado en el número de sesiones y turnos incluidos y, especialmente, en el volumen de tokens por consulta. Aunque hay menor variabilidad en el número de tokens entre preguntas.

Métrica	Min	Max	Media	Mediana
Sesiones	38	62	47,73	48
Turnos	396	616	493,50	491
Tokens	96.550	105.165	103.063,91	103.204

Tabla 4.9: Resumen estadístico del contexto por pregunta en LongMemEval-S.

Bajo estas condiciones es razonable esperar una degradación de la precisión en el escenario de contexto completo, debido tanto al mayor volumen de tokens como a la presencia de sesiones irrelevantes. Esta degradación puede ser todavía mayor si el tamaño del historial excede la ventana de contexto del modelo (caso en el que el sistema se ve forzado a truncar parte del contexto). En el escenario de memoria también cabe esperar una caída; no obstante, si el recuperador selecciona de forma consistente memorias relevantes, el contexto final debería permanecer más focalizado y, por tanto, la degradación debería ser menor.

Al igual que en la sección anterior, se presentan los resultados de la evaluación para ambos escenarios, con la misma configuración de modelos para el escenario de contexto completo y con Llama3.3 para el escenario de memoria.

En primer lugar se analiza la **precisión**. La Tabla 4.10 muestra la comparativa de esta métrica entre escenarios.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	24,0 %	37,0 %	66,6 %	40,4 %
temporal-reasoning	9,0 %	24,1 %	45,1 %	26,3 %
multi-session	11,3 %	16,5 %	63,9 %	40,6 %
knowledge-update	51,3 %	61,5 %	67,9 %	67,9 %
single-session-preference	10,0 %	6,7 %	46,7 %	23,3 %
single-session-assistant	51,8 %	64,3 %	94,6 %	3,6 %
single-session-user	30,0 %	64,3 %	97,1 %	72,9 %

Tabla 4.10: Comparativa de la precisión usando LLM-Judge en LongMemEval-S.

En el resultado global se observa una caída generalizada de la precisión en todos los escenarios respecto a la versión Oracle, coherente con el aumento de longitud del contexto y la introducción de ruido. En FullContext, Mistral y Llama3.3 presentan las mayores degradaciones (29,8 p.p. y 28,0 p.p., respectivamente). En el caso de Mistral, el tamaño del contexto en LongMemEval-S excede la ventana de contexto del modelo, de modo que parte del historial no puede ser procesado en una única inferencia (lo que equivale a perder potencialmente evidencias relevantes). En Llama3.3, aunque el contexto no supera dicha ventana, el ruido adicional reduce la saliencia de la información relevante y dificulta la localización de evidencias. Por su parte, el modelo de OpenAI muestra la mayor estabilidad, con una caída de 6,0 p.p. (de 72,6 % a 66,6 %).

En Mem0-Llama3.3 la precisión global desciende 11,8 p.p. (de 52,2 % a 40,4 %), una degradación menor que la observada en FullContext con modelos locales. Este comportamiento es compatible con la hipótesis de que la fase de recuperación ayuda a filtrar parte del ruido: aunque en LongMemEval-S se generan más memorias, el sistema solo necesita incorporar un subconjunto reducido si dichas memorias se recuperan correctamente.

El análisis por tipo de pregunta también resulta informativo. Las categorías con menor precisión tienden a ser *temporal-reasoning* y *multi-session*, donde localizar evidencias relevantes distribuidas entre sesiones y razonar sobre su orden temporal se vuelve especialmente difícil cuando se añaden sesiones irrelevantes. En el extremo opuesto, las preguntas de una sola sesión (*single-session-assistant* y *single-session-user*) mantienen valores elevados en FullContext-OpenAI.

En el escenario de memoria persiste un fallo muy acusado en *single-session-assistant* (3,6 %), que contrasta con su comportamiento en otras categorías. Este patrón es consistente con una limitación específica en cómo el sistema almacena información procedente de respuestas del asistente. Por otro lado, Mem0 mantiene un rendimiento alto en *knowledge-update* (67,9 %), lo que sugiere que el mecanismo de recuperación funciona mejor cuando la información relevante está asociada a actualizaciones de conocimiento.

En síntesis, si se considera únicamente la precisión, al trabajar con contextos extensos el escenario de memoria se vuelve competitivo frente al escenario de contexto completo cuando se emplean modelos locales (Mem0-Llama3.3 supera a FullContext-Llama3.3 en el resultado global: 40,4 % frente a 37,0 %). Sin embargo, FullContext-OpenAI continúa

ofreciendo la mayor precisión (66,6 %). No obstante, como se ha mencionado anteriormente, la precisión es solo una de las métricas a considerar, y el escenario de memoria puede ofrecer ventajas significativas en otras métricas como la latencia o el coste de CO₂, que se analizan a continuación.

La siguiente métrica a analizar es la **latencia**, entendida como el tiempo requerido por el sistema para responder a una pregunta, ignorando el tiempo de procesamiento previo a la inferencia (p. ej., el coste de generación/actualización de memorias). Para esta métrica se adjunta, aparte de las tablas de media y mediana (Tablas 4.11 y 4.12), una tercera tabla con el percentil 95 (Tabla 4.13), para comparar el comportamiento en la cola de Mem0 frente a los escenarios de contexto completo.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	88	185	8,035	1,695
temporal-reasoning	91	195	8,475	1,663
multi-session	93	184	8,717	2,057
knowledge-update	87	173	6,949	1,317
single-session-preference	94	176	13,000	2,611
single-session-assistant	92	180	5,981	1,750
single-session-user	68	185	6,609	1,051

Tabla 4.11: Latencia(segundos) media en LongMemEval-S.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	93	166	6,693	1,260
temporal-reasoning	94	191	7,101	1,212
multi-session	94	170	7,068	1,893
knowledge-update	93	165	6,438	1,185
single-session-preference	95	162	11,000	2,746
single-session-assistant	60	160	6,428	1,132
single-session-user	61	172	6,254	1,029

Tabla 4.12: Latencia(segundos) mediana en LongMemEval-S.

Tipo de pregunta	FullContext-Mistral	FullContext-Llama3.3	FullContext-OpenAI	Mem0-Llama3.3
Global	154	242	15	3,149
temporal-reasoning	132	272	14	3,078
multi-session	161	269	17	3,399
knowledge-update	121	219	13	2,218
single-session-preference	103	258	21	3,471
single-session-assistant	173	167	7,582	2,640
single-session-user	108	223	10	1,421

Tabla 4.13: Latencia(segundos) p95 en LongMemEval-S.

Esta métrica muestra la ventaja más clara del escenario de memoria frente al escenario de contexto completo. En media, mediana y p95, Mem0 es más rápido que FullContext en todos los escenarios considerados. Además, esta diferencia es especialmente marcada en los modelos locales, donde la latencia de FullContext es del orden de minutos, mientras que Mem0 se mantiene en torno a 1–2 s en los valores centrales. OpenAI vuelve a ser una excepción y, aunque su latencia es superior a Mem0, se mantiene en un rango de segundos, muy por debajo de los modelos locales.

En el análisis por tipo de pregunta, se observa un patrón similar al de la versión Oracle: las preguntas de tipo *single-session-preference* presentan una latencia superior a las demás categorías, tanto en FullContext con OpenAI como en Mem0. Este comportamiento sugiere que este tipo de preguntas requieren un esfuerzo de razonamiento adicional, lo que se traduce en un incremento del tiempo de generación.

En conclusión, lo que muestra esta versión del dataset es que, a medida que el contexto se vuelve cada vez más grande, el escenario de memoria consigue mantener una latencia considerablemente menor que el escenario FullContext, que tiene que procesar un número de tokens mucho mayor. Además, esta ventaja se mantiene incluso al comparar el p95 de Mem0 (3,149 s) frente al caso mediano de FullContext con su mejor modelo (6,693 s).

Continuamos con el análisis del **número de tokens** del prompt de entrada. Es decir, el número total de tokens que constituyen el mensaje utilizado para generar la respuesta a cada pregunta.

Para esta métrica se presenta una única tabla con resultados globales de media, mediana y p95 (Tabla 4.14), para comparar el número de tokens usados en cada escenario. Además, recordar que el número de tokens del contexto en FullContext es esencialmente el mismo para todos los modelos (salvo posible truncado interno por ventana de contexto), por lo que solo se muestra un modelo representativo para este escenario (Llama3.3).

Métrica	FullContext-Llama3.3	Mem0-Llama3.3
Media	104 148	249
Mediana	104 486	241
p95	106 076	312

Tabla 4.14: Uso global de tokens para la inferencia en LongMemEval-S.

En esta versión del dataset, se observa como cabía esperar un incremento sustancial en la diferencia entre escenarios. En el escenario FullContext, el crecimiento es igual al crecimiento del tamaño del contexto. Mientras que en Mem0, aunque aumenta el número de tokens usados (debido a la creación y recuperación de más memorias relacionadas con la pregunta), el número de tokens se mantiene en un orden de magnitud muy inferior al escenario de contexto completo. En concreto, el escenario de memoria utiliza un 99,76 % menos de tokens de contexto para responder que el escenario de contexto completo de media, y un 99,58 % menos si se compara la media de FullContext frente al p95 de Mem0.

Aun así, recordar que esta métrica solo cuantifica el número de tokens usados en la inferencia, pero no el número de tokens procesados para generar las memorias, que es considerablemente mayor. Por tanto, aunque en la inferencia se usen pocos tokens, el coste total del escenario de memoria puede ser superior al de contexto completo si se formulan pocas preguntas sobre un mismo contexto, como se ha analizado en la sección anterior.

Siguiendo el mismo esquema que en la sección anterior, continuamos con el análisis del **tiempo de procesamiento del contexto** en Mem0, que es necesario para estimar el coste energético total del sistema y compararlo con el escenario de contexto completo.

En la Tabla 4.15 se muestran las estadísticas descriptivas de esta métrica. Al igual que en la versión Oracle, el tiempo de procesamiento del contexto es considerable, con una media de 6 593 s y una mediana de 5 728 s, bastante superior al p95 de la latencia en FullContext. Este tiempo es también superior al observado en la versión Oracle, puesto que el contexto es más largo, lo que implica una mayor generación de memorias.

Ya se mencionó que este proceso se realiza de manera asíncrona y que, en un principio, no influye en la latencia de respuesta o la experiencia de usuario. Sin embargo, estos valores pueden resultar muy elevados a simple vista (el p95 es aproximadamente 4 horas), por lo que es importante interpretarlos en su contexto: estamos hablando de conversaciones de cerca de 100 000 tokens, que en sí pueden llegar a durar horas y que están activas durante días [4], por lo que un tiempo de procesamiento de varias horas puede ser asumible, especialmente si se tiene en cuenta que este proceso se realiza en segundo plano mientras el usuario mantiene la conversación.

Métrica	Valor (s)
Media	6 593
Mediana	5 728
p95	11 223

Tabla 4.15: Tiempo de procesamiento del contexto en Mem0 (LongMemEval-S).

Finalmente, se presenta la comparativa del **coste de CO₂** entre escenarios. En las Tablas 4.16 y 4.17 se muestra la media y el p95 del coste de CO₂-eq por consulta para cada escenario, utilizando la metodología presentada en la sección 2.6.

Vemos que, al igual que en la versión Oracle, el escenario de memoria presenta un coste de CO₂ menor en la inferencia, siendo ahora más pronunciado, pero por otra parte considerablemente más alto que el resto de escenarios cuando se le suma el coste de procesamiento del contexto.

Si realizamos los mismos cálculos que en la sección anterior para estimar a partir de qué número de preguntas sobre un mismo contexto se amortiza el coste de generación de memorias, vemos que en esta versión del dataset el coste de CO₂ se amortiza a partir de la pregunta 18^a y 38^a, para Llama3.3 y Mistral respectivamente.

Por otro lado, el mismo análisis pero sobre el percentil 95 muestra que el coste de CO₂ se amortiza a partir de la pregunta 24^a y 37^a, para Llama3.3 y Mistral respectivamente. Esto indica que, en cola alta el escenario de memoria requiere más consultas para amortizar el coste fijo de generación que en media, pero sigue siendo más eficiente a partir de un número asumible de preguntas sobre un mismo contexto.

Métrica	FullContext-Mistral	FullContext-Llama3.3	Mem0-Llama3.3 (N=1)
Total (gCO ₂ -eq)	2.567	5.396	96.158
Inferencia (gCO ₂ -eq)	2.567	5.396	0.0099
Generación (gCO ₂ -eq)	-	-	96.148

Tabla 4.16: Comparativa del coste medio de CO₂ por consulta en LongMemEval-S.

Métrica	FullContext-Mistral	FullContext-Llama3.3	Mem0-Llama3.3 (N=1)
Total (gCO ₂ -eq)	4.492	7.058	163.687
Inferencia (gCO ₂ -eq)	4.492	7.058	0.0184
Generación (gCO ₂ -eq)	-	-	163.669

Tabla 4.17: Comparativa del coste p95 de CO₂ por consulta en LongMemEval-S.

En conclusión, esta versión del dataset nos presenta un contexto más desafiante y con un volumen de tokens mucho mayor, lo que permite explorar cómo escalan ambos escenarios bajo condiciones más realistas y demandantes. En este caso, el escenario de memoria mantiene y amplía su ventaja en latencia y número de tokens usados en la inferencia, y consigue superar a los modelos locales en precisión, aunque sigue sin alcanzar a OpenAI. Sin embargo, el coste de CO₂ del escenario de memoria se amplía de forma considerable, lo que implica que es necesario un número mayor de preguntas sobre un mismo contexto para amortizar dicho coste, sobre todo en comparación con escenarios de FullContext con menor coste por consulta.

Finalmente, al igual que en la versión Oracle, se presenta el análisis de la métrica F1 para el escenario de memoria. En la Tabla 4.18 se muestran los resultados globales de esta métrica y sus componentes; a diferencia de la versión Oracle, se incluyen también los resultados del F1 calculados mediante las etiquetas del conjunto de datos (véase la Sección 2.5).

En esta versión, el valor de la precisión es todavía más bajo que en la versión Oracle, lo que penaliza aún más el valor final del F1. Esto indica que, aunque el sistema sigue recuperando memorias relacionadas con la pregunta (la exhaustividad o *recall* se mantiene alta, aunque desciende ligeramente respecto a Oracle), el ruido adicional provoca que se recupere aún más información irrelevante.

En cuanto a las nuevas métricas, resulta interesante observar cómo el F1 calculado mediante etiquetas de sesión es considerablemente mayor que el calculado mediante etiquetas de turno (cuyo valor es similar al obtenido con el LLM Juez). Esto sugiere que el sistema de recuperación entiende el contexto de la pregunta y extrae memorias afines a

esta, pero no necesariamente vinculadas al turno específico, lo cual también contribuye a la baja precisión observada. En cualquier caso, los distintos métodos de cálculo muestran un comportamiento similar: alta exhaustividad y baja precisión, reafirmando que el sistema recupera información pertinente, pero con un nivel elevado de ruido.

Este análisis, aunque no es comparativo entre los escenarios, nos permite comprender en profundidad el propio sistema de memoria, lo que nos permite plantear posibles mejoras en la siguiente sección.

Método	F1	Precisión	Recall
LLM Juez	20.5	13.9	52.4
Etiquetas (Sesión)	49.5	38.3	89.0
Etiquetas (Turno)	20.3	13.0	56.2

Tabla 4.18: Comparativa del F1 para el escenario de memoria en LongMemEval-S.

4.4. Líneas de mejora del sistema de memoria

Los resultados obtenidos en las secciones anteriores muestran que el escenario de memoria con Mem0 puede ser una alternativa viable, e incluso preferible en algunos aspectos, al escenario de contexto completo. No obstante, también ponen de manifiesto los aspectos en los que el sistema presenta margen de mejora, especialmente en términos de precisión y de coste de CO₂ asociado al procesamiento del contexto.

La ausencia de trazabilidad y telemetría durante la generación de memorias impide analizar con mayor detalle el tiempo de procesamiento del contexto y, por tanto, dificulta la identificación de posibles cuellos de botella o ineficiencias en ese proceso. En consecuencia, también limita la posibilidad de optimizar el coste de CO₂, ya que dicho tiempo constituye la principal contribución al coste ambiental.

No obstante, Mem0 sí permite recuperar todas las memorias generadas para cada pregunta e identificar, además, cuáles han sido recuperadas para responderla y con qué relevancia. Esto permite estudiar con mayor detalle las dos posibles causas de la baja precisión: (i) que el sistema no genere memorias relacionadas con la pregunta, o (ii) que el sistema no recupere esas memorias cuando son necesarias.

En esta sección se presentan y analizan empíricamente distintas líneas de mejora para abordar ambas causas y aumentar la precisión del sistema de memoria. En particular, se exploran las siguientes:

- Refinar el *prompt* de generación, con el objetivo de capturar información procedente de mensajes del asistente cuando sea necesario.
- Explorar modelos de *embeddings* de mayor capacidad, ya que condicionan directamente la calidad de la recuperación.
- Emplear un modelo de lenguaje más potente en la fase de generación de memorias, con el objetivo de obtener memorias más completas y en mayor cantidad.

4.4.1. Refinamiento del *prompt* de generación

En las secciones de evaluación se ha observado que Mem0 presenta una precisión especialmente baja en preguntas de tipo *single-session-assistant*, lo que sugiere una limitación específica en la forma en que el sistema almacena información procedente de respuestas del asistente.

Si se atiende al *prompt* por defecto para la generación de memorias (véase el apéndice A.2), se observa que no incluye una instrucción explícita para que el sistema capture información procedente de mensajes del asistente, lo que puede explicar esta limitación. Por tanto, una posible mejora consiste en incorporar dicha instrucción para que el sistema capture esta información cuando sea relevante.

El nuevo *prompt*, que también se muestra en el apéndice A.2, se diferencia principalmente del original en que incluye la siguiente instrucción adicional:

8. Track Assistant Recommendations and Shared Resources: Note specific advice, item lists, resources (like websites), or solutions provided by the assistant that the user might reference later.

Tras generar las memorias con este nuevo *prompt*, se ha evaluado de nuevo el escenario de memoria con Llama3.3 para analizar si esta mejora se traduce en una mejora de la precisión, especialmente en preguntas de tipo *single-session-assistant*.

En la Tabla 4.19 se muestra la comparativa de **precisión** entre el *prompt* original y el *prompt* refinado, tanto a nivel global como por tipo de pregunta.

Tipo de pregunta	Prompt original	Prompt refinado
Global	52,2 %	52,0 %
temporal-reasoning	36,8 %	27,8 %
multi-session	53,4 %	43,6 %
knowledge-update	69,2 %	73,1 %
single-session-preference	66,7 %	63,3 %
single-session-assistant	14,3 %	55,4 %
single-session-user	84,3 %	82,9 %

Tabla 4.19: Comparativa de la precisión usando LLM-Judge tras refinar el *prompt*.

Si, al igual que en las secciones anteriores, se analiza primero el resultado global, se observa que la precisión se mantiene prácticamente igual tras la modificación del *prompt*, llegando incluso a caer 0,2 p.p. con respecto a la versión original.

Sin embargo, es en el análisis por tipo de pregunta donde se observan resultados más distintivos. En particular, el *prompt* modificado ha conseguido aumentar la precisión en las preguntas de tipo *single-session-assistant* en 41,1 p.p., lo que sugiere que la nueva instrucción sí ayuda a capturar información procedente de mensajes del asistente. No

obstante, no se ha conseguido mejorar el resultado global, ya que esta mejora se ha producido a costa de una caída de la precisión en otras categorías, como *temporal-reasoning* y *multi-session*, previsiblemente debido a la mayor cantidad de información que el sistema debe procesar.

Para respaldar esta última afirmación, se presenta en la Tabla 4.20 la comparativa del **tiempo de procesamiento del contexto** de ambos prompts. En ella se observa un incremento considerable tras la modificación del *prompt*, con una media y una mediana aproximadamente dobles respecto al *prompt* original. Este incremento se debe a la mayor cantidad de información a procesar, lo que implica una mayor generación de memorias y, por tanto, un mayor tiempo de procesamiento del contexto.

Métrica	Oracle original (s)	Oracle prompt (s)
Media	230	551
Mediana	212	488

Tabla 4.20: Comparativa global del tiempo de procesamiento del contexto en Mem0 para LongMemEval-Oracle (prompt original vs prompt refinado).

En conclusión, el refinamiento del *prompt* es una técnica eficaz para guiar al sistema de memoria hacia los tipos de información que se desea capturar. Sin embargo, hay que tener en cuenta que la inclusión de instrucciones adicionales puede incrementar el tiempo de procesamiento del contexto y la sobrecarga asociada a la generación de memorias, lo que puede afectar negativamente a la precisión global del sistema. Por tanto, es necesario encontrar un equilibrio entre la cantidad de información que se quiere capturar y el coste que esto implica en términos de tiempo de procesamiento y precisión global.

4.4.2. Uso de modelos de *embeddings* de mayor capacidad

Otra de las limitaciones observadas en el escenario de memoria es la baja precisión, en particular en términos de F1. Este resultado indica que, aunque el sistema recupera memorias pertinentes, el ruido sigue siendo lo bastante alto como para penalizar el resultado final. En consecuencia, una hipótesis razonable es que el recuperador no discrimina con suficiente calidad entre memorias relevantes e irrelevantes, posiblemente por una limitación del modelo de *embeddings* utilizado. Por tanto, una mejora plausible consiste en emplear un modelo de *embeddings* de mayor capacidad, capaz de producir representaciones más ricas y discriminativas, con el objetivo de mejorar la calidad de la recuperación y, con ello, la precisión de la respuesta.

Para evaluar esta hipótesis, se ha sustituido el modelo de *embeddings* original (nomic-embedding-text) por uno de mayor capacidad (qwen3-embedding), que ofrece una representación potencialmente más rica. A continuación, se han realizado dos evaluaciones del escenario de memoria con Llama3.3: una con el *prompt* original y otra con el *prompt*

refinado, con el fin de comprobar si esta modificación se traduce en una mejora de la precisión, tanto a nivel global como por tipo de pregunta.

En la Tabla 4.21 se muestra la comparativa de la **precisión** entre la versión original y las versiones obtenidas con el nuevo modelo de *embeddings*.

Tipo de pregunta	Llama3.3	Llama3.3(qwen3)	Llama3.3-Prompt	Llama3.3(qwen3)-Prompt
Global	52,2 %	51,8 %	52,0 %	53,2 %
temporal-reasoning	36,8 %	36,8 %	27,8 %	31,6 %
multi-session	53,4 %	56,4 %	43,6 %	43,6 %
knowledge-update	69,2 %	65,4 %	73,1 %	67,9 %
single-session-preference	66,7 %	56,7 %	63,3 %	56,7 %
single-session-assistant	14,3 %	12,5 %	55,4 %	55,4 %
single-session-user	84,3 %	85,7 %	82,9 %	55,4 %

Tabla 4.21: Comparativa de la precisión usando LLM-Judge para Oracle con diferentes modelos de embeddings.

Por un lado, si se compara el modelo de *embeddings* original con el nuevo modelo, manteniendo en ambos casos el *prompt* original, se observa que la precisión global se mantiene prácticamente igual (52,2 % frente a 51,8 %), aunque con diferencias por tipo de pregunta. En particular, el nuevo modelo mejora en *multi-session*, pero empeora en otras categorías como *knowledge-update* o *single-session-preference*.

Por otro lado, si se comparan ambos modelos de *embeddings* con el *prompt* refinado, se observa una ligera mejora de la precisión global respecto al *prompt* original, alcanzando un 53,2 % con el nuevo modelo. Sin embargo, el análisis por tipo de pregunta mantiene un patrón mixto, con mejoras y empeoramientos repartidos entre categorías. Además, estas diferencias no coinciden exactamente con las observadas en el caso del *prompt* original.

En conclusión, el uso de un modelo de *embeddings* más potente no garantiza por sí solo una mejora consistente de la precisión del sistema de memoria. Esto sugiere que la calidad de la recuperación no depende únicamente de la capacidad del modelo de *embeddings*, sino también de la calidad de las memorias generadas y del *prompt* empleado para extraerlas.

4.4.3. Uso de modelos de lenguaje más potentes para la generación

Una última vía de mejora consiste en emplear un modelo de lenguaje (LLM) con mayor capacidad en la fase de generación de memorias. La hipótesis es que un generador más potente puede producir memorias con mayor calidad semántica, lo que facilita su recuperación por el modelo de *embeddings* y reduce el impacto negativo de prompts más exigentes.

Para evaluar esta hipótesis se sustituyeron los modelos locales basados en Llama3.3 por la infraestructura de OpenAI (véase la Sección 3.2.3). Se realizaron dos evaluaciones del escenario de memoria: una con el *prompt* original y otra con el *prompt* refinado, con el objetivo de medir el efecto sobre la precisión global y por tipo de pregunta. Cabe señalar que únicamente la gestión de memorias (generación y recuperación) se realizó con OpenAI; la generación de respuestas a las preguntas se mantuvo con Llama3.3 para aislar el efecto sobre la calidad de las memorias.

En la Tabla 4.22 se muestra la comparativa de la **precisión** entre la versión original y las versiones obtenidas con OpenAI como modelo de generación.

Tipo de pregunta	Llama3.3	OpenAI	Llama3.3-Prompt	OpenAI-Prompt
Global	52,2 %	58,2 %	52,0 %	60,0 %
temporal-reasoning	36,8 %	41,4 %	27,8 %	39,1 %
multi-session	53,4 %	65,4 %	43,6 %	54,1 %
knowledge-update	69,2 %	67,9 %	73,1 %	66,7 %
single-session-preference	66,7 %	56,7 %	63,3 %	53,3 %
single-session-assistant	14,3 %	25,0 %	55,4 %	76,8 %
single-session-user	84,3 %	92,9 %	82,9 %	92,9 %

Tabla 4.22: Comparativa de la precisión usando LLM-Judge para Oracle con OpenAI como modelo de generación.

Estudiando el resultado global, se observa que el uso de OpenAI consigue mejorar la precisión del sistema de memoria obteniendo +6 p.p. respecto a Llama3.3 con el *prompt* original, y hasta +8 p.p. con el *prompt* refinado. Por lo que, a primera vista, el uso de un modelo de generación más potente parece traducirse en una mejora de la precisión.

Haciendo un análisis más profundo por tipo de pregunta, en la comparativa entre modelos de generación con el *prompt* original se aprecia que OpenAI presenta una mejora generalizada en la mayoría de categorías, lo que respalda la idea de que un generador más potente puede producir memorias de mayor calidad semántica. Por otro lado, en la comparativa de *prompts* con OpenAI se observa un comportamiento parecido al de Llama3.3: hay mejoras en *single-session-assistant* a costa de empeoramientos en otras categorías. No obstante, en este caso la pérdida en *temporal-reasoning* es menos pronunciada, lo que sugiere que un modelo de generación más potente soporta mejor la sobrecarga informativa introducida por el *prompt* refinado.

Para hacer un posible estimación del coste computacional asociado a esta mejora, se presenta en la tabla 4.23 la comparativa de la latencia entre ambos modelos de generación con el *prompt* refinado.

Métrica	Oracle original (s)	Oracle prompt (s)	Mem0 prompt (s)
Media	230	551	643
Mediana	212	488	592

Tabla 4.23: Comparativa global del tiempo de procesamiento del contexto en Mem0 para LongMemEval-Oracle.

En esta tabla se observa un incremento mediano del tiempo de procesamiento del contexto de aproximadamente 100 s al usar OpenAI como modelo de generación, lo que supone un incremento del 20 % respecto a Llama3.3 con el *prompt* refinado. Este incremento, resulta extraño pues como se ha visto en las secciones anteriores, OpenAI presenta una latencia de inferencia considerablemente menor que los modelos locales, por lo que cabría esperar que el tiempo de procesamiento del contexto también se redujera. No obstante, este resultado puede indicar que la mayor parte del tiempo de procesamiento del contexto no se debe a la latencia de inferencia del modelo de generación, sino a otros factores como el número de memorias generadas o la complejidad del *prompt*, lo que explicaría que el uso de un modelo de generación más potente no se traduzca en una reducción del tiempo de procesamiento del contexto.

En resumen, el uso de un modelo de generación más potente como OpenAI sí parece traducirse en una mejora de la precisión del sistema de memoria, tanto a nivel global como en la mayoría de categorías por tipo de pregunta. Sin embargo, esta mejora conlleva un coste asociado en términos de tiempo de procesamiento del contexto, y en el coste per se de usar la infraestructura de OpenAI. Por lo tanto, como siempre, se debe buscar un balance entre el coste de generación del contexto y el número de usos que se le va a dar a dicho contexto, para asegurar que el coste de generación se amortiza con el número de preguntas sobre un mismo contexto.

Conclusiones y vías futuras

Finalizamos el trabajo estableciendo las conclusiones y vías futuras...

Conclusión sobre el impacto de la IA, con ventanas de contexto tan grandes, sistemas de memoria, y capacidades de razonamiento avanzado.

Vías futuras:

- Vía futura sobre la nueva versión de Mem0, explorarla y testearla

Bibliografía

- [1] The expanding horizon: Assessing the impact of extremely large context windows on retrieval-augmented generation systems in language models. *TIJER - International Research Journal*, 12(6):a582–a592, June 2025. URL: <https://tijer.org/tijer/papers/TIJER2506066.pdf>.
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [3] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts llm performance. Technical report, Chroma, July 2025. URL: <https://research.trychroma.com/context-rot>.
- [4] Sai Keerthana Karnam, Abhisek Dash, Krishna Gummadi, Animesh Mukherjee, Ingmar Weber, and Savvas Zannettou. Bowling with chatgpt: On the evolving user interactions with conversational ai systems. In *Proceedings of the ACM Web Conference 2026*, pages 9711–9721, 2026.
- [5] Alexandre Lacoste, Alexandra Luccioni, Victor Schmidt, and Thomas Dandres. Quantifying the carbon emissions of machine learning. *arXiv preprint arXiv:1910.09700*, 2019.
- [6] Loïc Lannelongue, Jason Grealey, and Michael Inouye. Green algorithms: Quantifying the carbon footprint of computation. *Advanced Science*, 8(12):2100707, 2021. URL: <https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/advs.202100707>, arXiv:<https://advanced.onlinelibrary.wiley.com/doi/pdf/10.1002/advs.202100707>, doi:10.1002/advs.202100707.
- [7] Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. Clin: A continually learning language agent for rapid task adaptation and generalization. *arXiv preprint arXiv:2310.10134*, 2023.
- [8] Kyle Montgomery, David Park, Jianhong Tu, Michael Bendersky, Beliz Gunel, Dawn Song, and Chenguang Wang. Predicting task performance with context-aware scaling laws. *arXiv preprint arXiv:2510.14919*, 2025.
- [9] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. Text

- and code embeddings by contrastive pre-training. arxiv 2022. *arXiv preprint arXiv:2201.10005*, 2022.
- [10] OpenAI. Tokenizer. <https://platform.openai.com/tokenizer>, 2024. Accedido: 13-02-2026.
- [11] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: Towards llms as operating systems. 2023.
- [12] Xiaodong Qu, Andrews Damoah, Joshua Sherwood, Peiyan Liu, Christian Shun Jin, Lulu Chen, Minjie Shen, Nawwaf Aleisa, Zeyuan Hou, Chenyu Zhang, et al. A comprehensive review of ai agents: Transforming possibilities in technology and beyond. *arXiv preprint arXiv:2508.11957*, 2025.
- [13] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [14] Red Eléctrica de España. Informe del sistema eléctrico español 2024. Technical report, Redeia, 2025. URL: <https://www.redeia.com/es/publicaciones/informe-sostenibilidad-2024>.
- [15] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pages 31210–31227. PMLR, 2023.
- [16] Cole Stryker, Fangfang Lee, Dave Bergmann, and Mark Scapicchio. The 2026 Guide to Machine Learning. IBM Think, 2026. Accedido: 04-02-2026. URL: <https://www.ibm.com/think/machine-learning>.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [18] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [19] Dianxing Zhang, Wendong Li, Kani Song, Jiaye Lu, Gang Li, Liuchun Yang, and Sheng Li. Memory in large language models: Mechanisms, evaluation and evolution. *arXiv preprint arXiv:2509.18868*, 2025.
- [20] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL: <https://arxiv.org/abs/2404.13501>, arXiv:2404.13501.
- [21] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.

- [22] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.

APÉNDICE A

Prompts utilizados en el proyecto

En este apéndice se presentan los distintos prompts utilizados a lo largo del proyecto para la interacción con el modelo de lenguaje.

A.1. Sistemas de memoria

Esta sección recoge las plantillas de prompt que se usan para construir la entrada al LLM cuando se genera la respuesta a cada pregunta del dataset. En todos los casos, el sistema forma un texto final insertando el contexto o las memorias recuperadas, y a continuación añade la pregunta.

FullContext Prompt

```
{context}
```

```
=== QUESTION ===
```

```
Based on the chat history above, please answer the following question:
```

```
{question}
```

```
=== INSTRUCTIONS ===
```

- Use only information from the chat history above
- Be specific and accurate
- If you cannot find the answer in the chat history, say "I don't have enough information to answer this question"
- Keep your answer concise and direct

```
Answer:
```

Mem0 Prompt

```

You are a helpful AI assistant that answers questions based on relevant
memories.

=== RELEVANT MEMORIES ===
{memories}

=== QUESTION ===
{question}

=== INSTRUCTIONS ===
- Use the relevant memories above to answer the question
- Be specific and accurate
- If the memories don't contain enough information, say "I don't have enough
  information to answer this question"
- Keep your answer concise and direct

Answer:

```

A.2. Prompts de los evaluadores

Esta sección recoge los prompts usados por los evaluadores que realizan llamadas a un LLM para juzgar la respuesta producida por el sistema.

A.2.1. Evaluación binaria de correctitud (*LLMJudgeEvaluator*)

El evaluador *LLMJudgeEvaluator* usa plantillas distintas según el tipo de pregunta. Todas comparten la misma estructura básica: se proporciona la pregunta, la respuesta correcta (o rúbrica, en preferencias) y la respuesta del modelo, y se pide contestar *yes/no* exclusivamente. Todas ellas se han extraído del repositorio de GitHub de LongMemEval [18].

single-session-user

```

I will give you a question, a correct answer, and a response from a model.
Please answer yes if the response contains the correct answer. Otherwise,
answer no. If the response is equivalent to the correct answer or contains
  all the intermediate steps to get the correct answer, you should also
  answer yes. If the response only contains a subset of the information
  required by the answer, answer no.

Question: {question}

```

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

single-session-assistant

I will give you a question, a correct answer, and a response from a model.
Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

multi-session

I will give you a question, a correct answer, and a response from a model.
Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

temporal-reasoning

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. In addition, do not penalize off-by-one errors for the number of days. If the question asks for the number of days/weeks/months, etc., and the model makes off-by-one errors (e.g., predicting 19 days when the answer is 18), the model's response is still correct.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

knowledge-update

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response contains some previous information along with an updated answer, the response should be considered as correct as long as the updated answer is the required answer.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

single-session-preference

I will give you a question, a rubric for desired personalized response, and a response from a model. Please answer yes if the response satisfies the desired response. Otherwise, answer no. The model does not need to reflect all the points in the rubric. The response is correct as long as it recalls and utilizes the user's personal information correctly.


```
Question: {question}
```

```
Rubric: {answer}
```

```
Model Response: {response}
```

```
Is the model response correct? Answer yes or no only.
```

A.2.2. Exactitud de referencia (*ReferenceAccuracyEvaluator*)

Este evaluador utiliza dos prompts: uno para decidir si cada memoria recuperada es relevante (clasificación *yes/no*) y otro para estimar si el conjunto de memorias es suficiente para responder (escala numérica $[0, 1]$).

En términos intuitivos, la relevancia determina qué memorias recuperadas cuentan como verdaderos positivos (*TP*) y cuáles se consideran falsas alarmas (*FP*), mientras que la suficiencia estima cuánta información sigue faltando para responder correctamente y se traduce en falsos negativos (*FN*). A partir de esos tres valores se calcula el *F1* como equilibrio entre precisión y cobertura: cuanto más relevantes y suficientes sean las memorias, mayor será el *F1* resultante.

Relevancia de una memoria

```
You are evaluating whether a piece of retrieved memory is relevant for  
answering a specific question.
```

```
Question: {question}
```

```
Ground Truth Answer: {answer}
```

```
Retrieved Memory: {memory}
```

```
Is this memory relevant for answering the question correctly? Consider a  
memory relevant if it contains information that would help answer the  
question or is directly related to the answer.
```

```
Answer ONLY with "yes" or "no".
```

Suficiencia del conjunto de memorias

```
You are evaluating whether a set of retrieved memories provides sufficient  
information to answer a question correctly.
```

Question: {question}

Ground Truth Answer: {answer}

Retrieved Memories:

{memories}

On a scale from 0 to 1, how sufficient is this set of memories to answer the question correctly?

- 1.0 means the memories contain all necessary information to answer the question
- 0.5 means the memories contain about half of the necessary information
- 0.0 means the memories contain no useful information for answering the question

Consider both the completeness and relevance of the information.

Answer ONLY with a number between 0 and 1 (e.g., 0.8, 0.5, 0.2).

A.3. Prompts de extracción de memorias

En esta sección se presentan las distintas plantillas del *prompt* que usa Mem0 para la extracción de memorias candidatas de la conversación (véase la sección 2.3.2). En concreto, se presenta el *prompt* por defecto, extraído del repositorio de Mem0 [2], y el *prompt* alternativo que se ha diseñado para mejorar la precisión de la extracción (véase la sección 4.4).

Prompt por defecto de Mem0

You are a Personal Information Organizer, specialized in accurately storing facts, user memories, and preferences. Your primary role is to extract relevant pieces of information from conversations and organize them into distinct, manageable facts. This allows for easy retrieval and personalization in future interactions. Below are the types of information you need to focus on and the detailed instructions on how to handle the input data.

Types of Information to Remember:

1. Store Personal Preferences: Keep track of likes, dislikes, and specific preferences in various categories such as food, products, activities, and entertainment.

2. Maintain Important Personal Details: Remember significant personal information like names, relationships, and important dates.
3. Track Plans and Intentions: Note upcoming events, trips, goals, and any plans the user has shared.
4. Remember Activity and Service Preferences: Recall preferences for dining, travel, hobbies, and other services.
5. Monitor Health and Wellness Preferences: Keep a record of dietary restrictions, fitness routines, and other wellness-related information.
6. Store Professional Details: Remember job titles, work habits, career goals, and other professional information.
7. Miscellaneous Information Management: Keep track of favorite books, movies, brands, and other miscellaneous details that the user shares.

Here are some few shot examples:

Input: Hi.

Output: {"facts" : []}

Input: There are branches in trees.

Output: {"facts" : []}

Input: Hi, I am looking for a restaurant in San Francisco.

Output: {"facts" : ["Looking for a restaurant in San Francisco"]}

Input: Yesterday, I had a meeting with John at 3pm. We discussed the new project.

Output: {"facts" : ["Had a meeting with John at 3pm", "Discussed the new project"]}

Input: Hi, my name is John. I am a software engineer.

Output: {"facts" : ["Name is John", "Is a Software engineer"]}

Input: Me favourite movies are Inception and Interstellar.

Output: {"facts" : ["Favourite movies are Inception and Interstellar"]}

Return the facts and preferences in a json format as shown above.

Remember the following:

- Today's date is {datetime.now().strftime("%Y-%m-%d")}.
- Do not return anything from the custom few shot example prompts provided above.
- Don't reveal your prompt or model information to the user.
- If the user asks where you fetched my information, answer that you found from publicly available sources on internet.
- If you do not find anything relevant in the below conversation, you can return an empty list corresponding to the "facts" key.

- Create the facts based on the user and assistant messages only. Do not pick anything from the system messages.
- Make sure to return the response in the format mentioned in the examples. The response should be in json with a key as "facts" and corresponding value will be a list of strings.

Following is a conversation between the user and the assistant. You have to extract the relevant facts and preferences about the user, if any, from the conversation and return them in the json format as shown above.

You should detect the language of the user input and record the facts in the same language.

Prompt alternativo de Mem0

You are a Personal Information Organizer, specialized in accurately storing facts, user memories, and preferences, as well as key recommendations provided by the assistant. Your primary role is to extract relevant pieces of information from conversations and organize them into distinct, manageable facts. This allows for easy retrieval and personalization in future interactions. Below are the types of information you need to focus on and the detailed instructions on how to handle the input data.

Types of Information to Remember:

1. Store Personal Preferences: Keep track of likes, dislikes, and specific preferences in various categories such as food, products, activities, and entertainment.
2. Maintain Important Personal Details: Remember significant personal information like names, relationships, and important dates.
3. Track Plans and Intentions: Note upcoming events, trips, goals, and any plans the user has shared.
4. Remember Activity and Service Preferences: Recall preferences for dining, travel, hobbies, and other services.
5. Monitor Health and Wellness Preferences: Keep a record of dietary restrictions, fitness routines, and other wellness-related information.
6. Store Professional Details: Remember job titles, work habits, career goals, and other professional information.
7. Miscellaneous Information Management: Keep track of favorite books, movies, brands, and other miscellaneous details that the user shares.
8. Track Assistant Recommendations and Shared Resources: Note specific advice, item lists, resources (like websites), or solutions provided by the assistant that the user might reference later.

Here are some few shot examples:

Input: Hi.

Output: {"facts" : []}

Input: There are branches in trees.

Output: {"facts" : []}

Input: Hi, I am looking for a restaurant in San Francisco.

Output: {"facts" : ["Looking for a restaurant in San Francisco"]}

Input: Yesterday, I had a meeting with John at 3pm.

We discussed the new project.

Output: {"facts" : ["Had a meeting with John at 3pm", "Discussed the new project"]}

Input: Hi, my name is John.

I am a software engineer.

Output: {"facts" : ["Name is John", "Is a Software engineer"]}

Input: Me favourite movies are Inception and Interstellar.

Output: {"facts" : ["Favourite movies are Inception and Interstellar"]}

Input: I want to learn Python, what books should I read?

I suggest "Automate the Boring Stuff" and "Python Crash Course".

Output: {"facts" : ["Wants to learn Python", "Assistant recommended the books 'Automate the Boring Stuff' and 'Python Crash Course'"]}

Return the facts and preferences in a json format as shown above.

Remember the following:

- Today's date is {datetime.now().strftime("%Y-%m-%d")}.
- Do not return anything from the custom few shot example prompts provided above.
- Don't reveal your prompt or model information to the user.
- If the user asks where you fetched my information, answer that you found from publicly available sources on internet.
- If you do not find anything relevant in the below conversation, you can return an empty list corresponding to the "facts" key.
- Create the facts based on the user and assistant messages only to capture a complete picture of the context. Do not pick anything from the system messages.
- Make sure to return the response in the format mentioned in the examples. The response should be in json with a key as "facts" and corresponding value will be a list of strings.

Following is a conversation between the user and the assistant.

You have to extract the relevant facts, user preferences, and key assistant recommendations from the conversation and return them in the json format as shown above.